



Escuela
Politécnica
Superior

Resolución de problemas de la Olimpiada Lingüística



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:

Alicja Wiktorja Hyska

Tutor/es:

Juan Antonio Pérez Ortiz

Junio 2026



Universitat d'Alacant
Universidad de Alicante

Resolución de problemas de la Olimpiada Lingüística

Aplicación de modelos de inteligencia artificial

Autor

Alicja Wiktoria Hyska

Tutor/es

Juan Antonio Pérez Ortiz

Lenguajes y Sistemas Informáticos



Grado en Ingeniería Informática



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Junio 2026

Preámbulo

Este trabajo surge del interés por explorar los límites del razonamiento lingüístico en los modelos de lenguaje actuales. El objetivo es diseñar y evaluar estrategias de prompting que permitan a estos modelos resolver puzzles de olimpiadas lingüísticas en idiomas nunca vistos durante su entrenamiento. El trabajo se enmarca en el área de procesamiento de lenguas de baja densidad de recursos y ha sido realizado bajo la supervisión de Juan Antonio Pérez Ortiz en el Grado de Ingeniería Informática de la Universidad de Alicante.

Agradecimientos

Quisiera agradecer, en primer lugar, a mi tutor, Juan Antonio Pérez Ortiz, por su orientación, sus consejos y su disponibilidad durante el desarrollo de este Trabajo de Fin de Grado. Su ayuda ha sido fundamental para dar forma al proyecto y para afrontar las distintas decisiones que han ido surgiendo a lo largo del proceso.

También quiero dar las gracias a mi familia, por su apoyo constante durante todos estos años. Su confianza, paciencia y ánimo han sido esenciales no solo durante la realización de este trabajo, sino a lo largo de toda mi etapa universitaria.

Este trabajo nace también de un interés personal por los idiomas y por la forma en que las lenguas permiten entender distintas culturas. Haber vivido en diferentes países me ha permitido acercarme a varios idiomas desde una perspectiva práctica y humana, y ha despertado en mí una curiosidad especial por la traducción y el razonamiento entre lenguas. Esa experiencia personal ha influido directamente en la elección de este tema, centrado en la resolución de problemas de olimpiadas lingüísticas mediante modelos de inteligencia artificial.

Por último, quiero agradecer a todas las personas que, de una forma u otra, me han acompañado durante este camino: compañeros, amigos y profesores que han contribuido a mi formación y me han ayudado a llegar hasta aquí.

El límite de mi lenguaje es el límite de mi mundo.

Ludwig Wittgenstein.

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Contexto y estado del problema	2
1.3. Estructura del documento	2
2. Marco Teórico y Estado de la Cuestión	5
2.1. Lenguas de baja densidad de recursos	5
2.2. Razonamiento metalingüístico	6
2.3. Benchmarks de olimpiadas lingüísticas para LLMs	6
2.3.1. PuzzLing Machines	7
2.3.2. modeLing	7
2.3.3. Linguini	7
2.3.4. LingOly	8
2.3.5. Comparación entre los principales benchmarks	8
2.4. Estrategias de prompting para razonamiento lingüístico	9
2.4.1. Aprendizaje en contexto con ejemplos	9
2.4.2. Razonamiento paso a paso	10
2.4.3. Prompting analógico	10
2.4.4. Gramática en contexto	10
2.5. Análisis de las dificultades de los LLMs en puzzles lingüísticos	11
2.6. Síntesis y posicionamiento del trabajo	11
3. Objetivos	15
3.1. Objetivo principal	15
3.2. Objetivos específicos	15
3.3. Restricciones y condicionantes	16
4. Metodología y datos	17
4.1. Dataset: Linguini	17
4.1.1. Criterios de selección	17
4.1.2. Puzzles seleccionados	17
4.1.3. Formato de los datos	18
4.2. Modelos evaluados	18
4.3. Estrategias de prompting	19
4.3.1. Instrucción metalingüística base	19
4.3.2. M0: Verificación	19
4.3.3. M1: Baseline	19
4.3.4. M2: Incremental por combinaciones	20
4.3.5. M3: Step-by-step lingüístico	20

4.3.6.	M4: Inspiración humana	21
4.3.7.	M5: Autocorrección iterativa	21
4.4.	Sistema experimental	21
4.4.1.	Extracción de traducciones.	21
4.4.2.	Parámetros de ejecución.	22
4.5.	Métricas de evaluación	22
4.6.	Análisis cualitativo	22
4.6.1.	Protocolo de resolución humana	23
4.6.2.	Proceso de resolución observado	23
4.6.3.	Comparación con el modelo	23
5.	Desarrollo del sistema	25
5.1.	Arquitectura general	25
5.1.1.	Formato de entrada	27
5.1.2.	Formato del log JSON	27
5.2.	Diseño de los prompts	28
5.2.1.	Instrucción metalingüística base	28
5.2.2.	Formato de salida exigido	29
5.2.3.	Prompt de M1 (baseline)	29
5.2.4.	Prompt de M2 (incremental)	30
5.2.5.	Prompt de M3 (step-by-step)	31
5.2.6.	Prompt de M4 (inspiración humana)	31
5.2.7.	Prompt de M5 (autocorrección)	32
5.3.	Extracción automática de traducciones	32
5.3.1.	Patrones de salida observados	33
5.3.2.	Arquitectura multicapa del extractor	33
5.3.3.	Filtros de descarte	35
5.3.4.	Normalización de las traducciones	35
5.3.5.	Gestión de respuestas incompletas	36
5.3.6.	Validación del extractor	36
5.4.	Construcción del conjunto de datos	37
5.4.1.	Extracción desde el JSONL de Linguini	37
5.4.2.	Criterios de selección	38
5.4.3.	Validación de los puzzles convertidos	39
5.5.	Problemas encontrados y soluciones	39
5.5.1.	Limitaciones de las APIs y los proveedores	40
5.5.2.	Problemas de implementación y extracción	41
5.5.3.	Problemas relacionados con las métricas	41
5.5.4.	Limitaciones que permanecen abiertas	41
6.	Resultados (Con ejemplos de gráficos)	45
6.1.	Diagramas	45
6.2.	Gráficas	46
6.2.1.	Línea	46
6.2.2.	Barras	47
6.2.3.	Polar	48

6.3. Importados de MATLAB	49
6.4. Ejemplo avanzado	50
7. Conclusiones (Con ejemplos de matemáticas)	53
7.1. Matemáticas	53
Bibliografía	57
A. Anexo I	61
B. Páginas horizontales	63
C. Importar PDF	67

Índice de figuras

5.1.	Flujo general del sistema experimental.	26
6.1.	Diagrama realizado en latex con Tikz.	46
6.2.	Gráfica sencilla.	47
6.3.	OP/S003	47
6.4.	Gráfica barras.	48
6.5.	Directividad normalizada del altavoz (0 dBV en el eje).	49
6.6.	Ejemplo de gráfica obtenida con matlab2tikz.	50
6.7.	Amplitud de la aceleración en el modo número 8.	50
6.8.	Señal realizada con Tikz, sin imágenes.	51

Índice de tablas

2.1.	Comparación de los principales benchmarks de olimpiadas lingüísticas para LLMs.	9
4.1.	Puzzles utilizados en los experimentos. Dir.: dirección de traducción ($\rightarrow$ EN = lengua desconocida al inglés; EN $\rightarrow$ = inglés a lengua desconocida).	18
4.2.	Modelos evaluados	18
5.1.	Principales patrones detectados en las respuestas de los modelos.	33
5.2.	Principales incidencias relacionadas con las APIs utilizadas.	40
5.3.	Principales problemas detectados durante la extracción de respuestas. . .	42
5.4.	Problemas detectados durante el cálculo e interpretación de métricas. . .	43

Índice de Códigos

5.1. Flujo principal de ejecución de un experimento	25
5.2. Estructura simplificada de un puzzle en formato JSON	27
5.3. Estructura simplificada del log de resultados	28
5.4. Lógica principal del extractor de traducciones	34
5.5. Filtros utilizados para descartar falsos positivos	35
5.6. Normalización básica de una traducción extraída	35
5.7. Penalización de traducciones ausentes	36
5.8. Metadatos asociados a la extracción	36
5.9. Estructura simplificada de un puzzle convertido	37
5.10. Validación básica de un puzzle convertido	39
5.11. Reintentos automáticos tras alcanzar el límite de tasa	40
6.1. Ejemplo de llamada a matlab2tikz	49

1. Introducción

1.1. Motivación

El lenguaje es una de las capacidades más complejas y distintivas de la especie humana. En el mundo existen miles de lenguas vivas (Eberhard y cols., 2020), pero la gran mayoría dispone de una presencia digital muy reducida, en muchos casos apenas existen corpus textuales, traducciones paralelas o herramientas de procesamiento automático. Esta escasez de información hace que los sistemas de inteligencia artificial convencionales, que dependen de grandes volúmenes de datos de entrenamiento, sean prácticamente inútiles para trabajar con ellas.

Sin embargo, los seres humanos somos capaces de aprender a leer, escribir y traducir en un idioma completamente desconocido a partir de un número reducido de ejemplos, utilizando nuestra *competencia metalingüística*. En otras palabras, las personas pueden identificar el vocabulario, reconocer patrones morfológicos y formular reglas gramaticales únicamente comparando palabras y frases. Esta capacidad, que demostramos cada vez que resolvemos un problema de la Olimpiada Lingüística, plantea una pregunta fundamental para la inteligencia artificial: ¿pueden los modelos de lenguaje de gran escala hacer lo mismo?

La Olimpiada Internacional de Lingüística (IOL) propone anualmente problemas del tipo *piedra Rosetta*: se presenta al participante un conjunto de frases en una lengua completamente desconocida junto con sus traducciones, y se le pide que deduzca las reglas del idioma y las aplique para traducir nuevas frases. Estos problemas son autocontenidos y requieren razonamiento explícito, no memorización. Dado que toda la información de los problemas está en el enunciado, estos son un banco de pruebas ideal para medir si un sistema de inteligencia artificial posee razonamiento metalingüístico genuino, o al menos si es capaz de reproducir este tipo de razonamiento.

En este trabajo vamos a partir de la hipótesis de que es posible diseñar estrategias de *prompting* que permitan a los modelos de lenguaje resolver estos problemas de forma progresiva, construyendo explícitamente un diccionario y un conjunto de reglas gramaticales a partir de los ejemplos proporcionados, sin recibir ninguna pista sobre la lengua en cuestión. Si esta hipótesis se confirma, las implicaciones son significativas: abriría la puerta a utilizar modelos de lenguaje para procesar, documentar y analizar miles de lenguas del mundo para las que no existe ni existirá suficiente cantidad de datos de entrenamiento. En caso contrario, estudiaremos y evidenciaremos hasta qué punto los modelos actuales pueden reproducir un proceso de análisis similar al que sigue una persona al enfrentarse a una lengua que no conoce.

Cabe mencionar que la motivación del proyecto también está relacionada con un interés personal por los idiomas y por la forma en que distintas lenguas expresan una misma realidad mediante estructuras muy diferentes. Desde esta perspectiva, los problemas de olimpiadas lingüísticas ofrecen un entorno especialmente adecuado para combinar el análisis lingüístico con el estudio de modelos de inteligencia artificial.

1.2. Contexto y estado del problema

Durante los últimos años, los modelos de lenguaje de gran escala han demostrado capacidades notables en tareas como la generación de texto, la traducción automática o la respuesta a preguntas. Una de sus características más relevantes es el aprendizaje en contexto, que es la posibilidad de realizar una tarea a partir de ejemplos incluidos directamente en el prompt, sin necesidad de modificar los parámetros del modelo.

Esta propiedad permite plantear una pregunta especialmente relevante para las lenguas de baja densidad de recursos: ¿puede un modelo deducir patrones lingüísticos a partir de unos pocos ejemplos, incluso cuando no dispone de información suficiente para apoyarse únicamente en los datos aprendidos durante su entrenamiento?

En los últimos años se han propuesto varios conjuntos de evaluación basados en problemas de olimpiadas lingüísticas. Entre ellos se encuentra Linguini, que reúne problemas de distintas lenguas y permite evaluar la capacidad de los modelos para resolver tareas como la traducción, la identificación de patrones morfológicos o la transliteración numérica. Los resultados publicados hasta el momento muestran que estos problemas siguen siendo difíciles para los modelos actuales. Aunque algunos sistemas obtienen respuestas parcialmente correctas, su rendimiento se mantiene por debajo del nivel humano en muchos tipos de ejercicios, especialmente cuando las lenguas presentan una morfología compleja o estructuras alejadas de las más frecuentes en los datos de entrenamiento.

Distintos trabajos han estudiado formas de mejorar estos resultados mediante estrategias de prompting más elaboradas. En lugar de solicitar directamente una traducción final, estas estrategias guían al modelo para que identifique vocabulario, deduzca reglas y revise sus hipótesis antes de responder.

Este Trabajo de Fin de Grado se sitúa en esa línea de investigación. Su objetivo es diseñar, implementar y comparar varias estrategias de prompting para la resolución de problemas de olimpiadas lingüísticas. En particular, se estudiará si una presentación incremental de los ejemplos permite al modelo construir conocimiento de forma progresiva y corregir hipótesis incorrectas a medida que recibe nueva información.

1.3. Estructura del documento

El presente documento se organiza en los siguientes capítulos:

El **Capítulo 2** presenta el estado de la cuestión, revisando los fundamentos teóricos del razonamiento metalingüístico, los principales benchmarks de olimpiadas lingüísticas para LLMs y las estrategias de prompting más relevantes de la literatura.

El **Capítulo 3** define el objetivo principal del trabajo y los objetivos específicos que se derivan de él, así como las restricciones y condicionantes del proyecto.

El **Capítulo 4** describe la metodología del trabajo: los puzzles utilizados, los modelos evaluados, las tres estrategias de prompting diseñadas, el sistema experimental implementado y las métricas de evaluación empleadas.

El **Capítulo 5** detalla la implementación del sistema experimental y las decisiones adoptadas durante su desarrollo.

El **Capítulo 6** presenta y analiza los resultados experimentales obtenidos, incluyendo la comparación con los valores de referencia del benchmark LINGUINI y el análisis cualitativo

comparando el razonamiento del modelo con el humano.

Por último, el **Capítulo 7** recoge las conclusiones del trabajo, sus limitaciones y las líneas de trabajo futuro que se abren a partir de los resultados obtenidos.

2. Marco Teórico y Estado de la Cuestión

Este capítulo revisa el estado del arte en los campos que confluyen en este trabajo. En primer lugar, se describe la desigual disponibilidad de recursos digitales entre las lenguas del mundo y sus consecuencias para el procesamiento del lenguaje natural. A continuación, se introduce el concepto de razonamiento metalingüístico y su relación con los problemas de olimpiadas lingüísticas. Finalmente, se revisan los principales conjuntos de evaluación y estrategias de prompting utilizados para estudiar este tipo de razonamiento en modelos de lenguaje de gran escala.

2.1. Lenguas de baja densidad de recursos

Se estima que existen entre 6.000 y 7.000 lenguas vivas en el mundo (Eberhard y cols., 2020), pero la inmensa mayoría de ellas carecen de los recursos digitales necesarios para desarrollar sistemas de procesamiento automático del lenguaje natural (PLN). Las lenguas que se encuentran en esta situación suelen denominarse lenguas de baja densidad de recursos (low-resource languages). No se trata de una categoría completamente cerrada, ya que la disponibilidad de recursos puede variar considerablemente de una lengua a otra: mientras algunas cuentan con textos sin anotar, pero carecen de corpus paralelos o herramientas lingüísticas, otras apenas tienen presencia escrita o digital.

Joshi y cols. (2020) cuantifican esta desigualdad mostrando que más del 88% de las lenguas del mundo apenas tienen representación en los conjuntos de datos utilizados para entrenar los modelos actuales. El estudio de dichas lenguas plantea uno de los retos más relevantes del PLN contemporáneo.

La escasez de datos tiene consecuencias directas en el rendimiento de los sistemas de traducción automática, reconocimiento de voz y análisis morfosintáctico para estas lenguas. Los modelos de lenguaje actuales, entrenados mayoritariamente sobre texto en inglés y otras lenguas de alta densidad, presentan un rendimiento muy inferior cuando se enfrentan a lenguas con poca presencia en sus datos de entrenamiento. Este desequilibrio no es únicamente un problema técnico: tiene implicaciones directas para la preservación del patrimonio lingüístico y cultural de comunidades hablantes de lenguas minorizadas.

Una aproximación prometedora para abordar este problema es el aprendizaje en contexto (*in-context learning*): la capacidad de los modelos de lenguaje de gran escala para realizar tareas nuevas a partir de unos pocos ejemplos proporcionados directamente en el prompt, sin necesidad de reentrenamiento. Esta capacidad abre la posibilidad de trabajar con lenguas para las que no existen datos de entrenamiento suficientes, siempre que el modelo sea capaz de razonar sobre los patrones lingüísticos presentes en los ejemplos.

2.2. Razonamiento metalingüístico

El concepto de *competencia metalingüística* hace referencia a la capacidad de razonar sobre el lenguaje como objeto de estudio, más allá del uso comunicativo cotidiano. Un hablante con alta competencia metalingüística es capaz de identificar patrones morfológicos en una lengua desconocida, deducir reglas gramaticales a partir de ejemplos y aplicarlas a nuevas frases, sin necesidad de haber aprendido previamente ese idioma.

Esta capacidad es precisamente la que explotan los problemas de la Olimpiada Lingüística. A los participantes se les presenta un conjunto de frases en una lengua completamente desconocida junto con sus traducciones al inglés, y deben deducir las reglas del idioma para traducir nuevas frases. Toda la información necesaria para resolver los problemas está en el enunciado y están diseñados para lenguas de baja densidad de recursos, lo que minimiza la posibilidad de que los participantes tengan conocimiento previo del idioma (Derzhanski y Payne, 2010).

Este formato permite estudiar una cuestión relevante para los modelos de lenguaje de gran escala: hasta qué punto son capaces de reproducir un proceso de inferencia lingüística a partir de pocos datos. La dificultad no consiste únicamente en generar una traducción plausible, sino en extraer patrones, mantener hipótesis coherentes y aplicarlas de forma sistemática.

Conviene distinguir entre el comportamiento observable del modelo y la competencia lingüística humana. Cuando un modelo resuelve correctamente un problema, no puede concluirse automáticamente que razone de la misma forma que una persona. Sin embargo, el análisis de sus respuestas, de sus errores y de las reglas que formula permite estudiar hasta qué punto reproduce algunos de los pasos característicos del razonamiento metalingüístico.

Los problemas de olimpiadas lingüísticas constituyen, por tanto, un entorno especialmente adecuado para esta evaluación. Requieren trabajar con pocos ejemplos, reducen la utilidad de una traducción basada únicamente en asociaciones superficiales y permiten comparar de forma detallada el resultado final con el proceso de deducción seguido para alcanzarlo.

2.3. Benchmarks de olimpiadas lingüísticas para LLMs

La evaluación del razonamiento lingüístico en modelos de lenguaje de gran escala ha dado lugar a varios conjuntos de datos basados en problemas de olimpiadas lingüísticas. Estos benchmarks comparten una idea central: presentar al modelo un número reducido de ejemplos en una lengua poco conocida y comprobar si es capaz de extraer patrones, formular reglas y aplicarlas a nuevas expresiones.

Este tipo de evaluación resulta especialmente adecuado para estudiar el aprendizaje en contexto. A diferencia de otras tareas lingüísticas, el objetivo no es medir cuánto conocimiento previo posee el modelo sobre una lengua concreta, sino analizar su capacidad para deducir regularidades a partir de la información incluida en el propio enunciado.

A continuación se presentan los principales trabajos relacionados con mi Trabajo de Fin de Grado.

2.3.1. PuzzLing Machines

El trabajo de Şahin y cols. (2020) constituye uno de los primeros intentos sistemáticos de utilizar problemas de olimpiadas lingüísticas para evaluar sistemas automáticos. Los autores presentan un conjunto de siete patrones de problemas de tipo piedra Rosetta extraídos de la IOL (correspondientes a 81 lenguas de 39 familias distintas), y evalúan la capacidad de modelos de traducción automática neural para resolverlos.

Cabe destacar que aunque estos problemas adoptan formalmente la apariencia de una tarea de traducción, presentan diferencias importantes con respecto a la traducción automática convencional. El sistema no dispone de grandes corpus paralelos, sino únicamente de un conjunto mínimo de ejemplos cuidadosamente seleccionado. A partir de ellos debe extraer un pequeño diccionario, identificar reglas gramaticales y reconocer posibles fenómenos morfológicos o fonológicos antes de traducir nuevas expresiones.

Los experimentos realizados por Şahin y cols. muestran que los modelos de traducción automática evaluados encuentran grandes dificultades para resolver estos problemas de forma robusta. El trabajo pone de manifiesto que aprender asociaciones estadísticas a partir de grandes cantidades de texto no es suficiente cuando la tarea exige deducir reglas a partir de pocos ejemplos.

2.3.2. modeLing

Chi y cols. (2024) presentan modeLing, un dataset construido manualmente con problemas similares a los de la IOL, diseñado específicamente para evitar la contaminación de datos de entrenamiento. A diferencia de los benchmarks basados directamente en el corpus de la IOL, los problemas de modeLing son más originales, lo que garantiza que los modelos evaluados no puedan haberlos memorizado durante el preentrenamiento.

Más concretamente, modeLing utiliza 48 problemas creados expresamente para el trabajo, basados en 19 lenguas con una presencia digital extremadamente reducida. En total, el conjunto incluye 272 preguntas. Los problemas evalúan distintos aspectos del razonamiento lingüístico, como el orden de palabras, la posesión, la semántica y la morfología nominal o adjetival.

Además, los autores realizan experimentos específicos para comprobar posibles fugas de datos y no encuentran evidencias significativas de contaminación en los modelos analizados.

El dataset ha sido utilizado como base de comparación en trabajos posteriores.

2.3.3. Linguini

Sánchez y cols. (2024) presentan Linguini, el benchmark que constituye la referencia central de este trabajo. Linguini está compuesto por 894 preguntas agrupadas en 160 problemas extraídos del corpus de la IOL, cubriendo 75 lenguas de baja densidad de recursos. Los problemas se clasifican en tres categorías:

1. Transducción de secuencias (sequence transduction): el modelo debe transformar una secuencia en otra, por ejemplo, traduciendo una frase o relacionando expresiones equivalentes.
 2. Relleno de huecos (fill-in-blanks): el modelo debe completar formas lingüísticas aplicando reglas morfológicas o fonológicas deducidas a partir de los ejemplos.
-

3. Transliteración numérica (number transliteration): el modelo debe convertir expresiones numéricas entre su representación textual y su representación mediante cifras.

Los resultados de la evaluación son reveladores: ninguno de los modelos analizados supera el 25% de exactitud. El mejor modelo propietario, Claude 3 Opus, alcanza un 24,05% de exactitud y un chrF++ de 63,96 en configuración zero-shot. El mejor modelo de código abierto, Llama 3 70B, obtiene un 8,84% de exactitud y un chrF++ de 37,25. Estos valores contrastan con el rendimiento humano en competición, que supera habitualmente el 60%.

Un hallazgo especialmente relevante del paper es el experimento de ablación sin contexto: al eliminar las frases de entrenamiento del prompt, el rendimiento de todos los modelos cae drásticamente, lo que confirma que los modelos resuelven los puzzles por razonamiento inductivo sobre el contexto proporcionado, no por memorización del idioma durante el preentrenamiento.

2.3.4. LingOly

Bean y cols. (2024) presentan LingOly, el benchmark más exhaustivo del campo hasta la fecha, publicado en NeurIPS 2024. LingOly cubre más de 90 lenguas (incluyendo algunas extintas) y contiene 1.133 problemas organizados en seis categorías lingüísticas (fonología, morfología, sintaxis, semántica, sistemas numéricos y problemas compuestos) y cinco niveles de dificultad humana. Esta clasificación permite realizar análisis más detallados y estudiar cómo varía el rendimiento de los modelos cuando aumenta la complejidad del razonamiento requerido.

Los autores evalúan once modelos de lenguaje y comparan sus resultados con un baseline sin contexto. Los experimentos muestran que los modelos encuentran dificultades especialmente importantes en los problemas de mayor nivel. Incluso el mejor sistema evaluado alcanza únicamente un 38,7 % de exactitud en los ejercicios más complejos, aunque mejora en 24,7% respecto a la versión sin contexto.

Los resultados también muestran una tendencia general: los modelos propietarios de mayor tamaño suelen superar a los modelos abiertos, y el rendimiento tiende a ser superior en lenguas con una mayor disponibilidad de recursos digitales.

Aunque LingOly no se utiliza directamente en los experimentos de este trabajo, su inclusión en el estado de la cuestión resulta relevante porque confirma que las limitaciones observadas en Linguini no son aisladas. Incluso cuando se amplía la variedad de problemas y se estudian modelos más recientes, el razonamiento lingüístico a partir de pocos ejemplos sigue planteando dificultades importantes, y es una afirmación que nos sirve de gran apoyo.

2.3.5. Comparación entre los principales benchmarks

Los cuatro conjuntos de evaluación analizados responden a objetivos complementarios. PuzzLing Machines establece el problema inicial y muestra las limitaciones de los sistemas convencionales de traducción automática. modeLing se centra en reducir el riesgo de contaminación mediante problemas creados específicamente para la evaluación. Linguini amplía considerablemente la escala del análisis y proporciona el marco experimental utilizado en este trabajo. Por último, LingOly incorpora una mayor variedad de formatos y niveles de dificultad. Como se resume en la Tabla 2.1, los cuatro benchmarks responden a objetivos complementarios.

En conjunto, estos trabajos muestran que los problemas de olimpiadas lingüísticas constituyen una herramienta útil para estudiar la capacidad de los modelos de lenguaje para generalizar a partir de pocos ejemplos. También ponen de manifiesto que el rendimiento sigue siendo limitado y que la forma de presentar la información al modelo puede tener una influencia importante en los resultados.

Tabla 2.1: Comparación de los principales benchmarks de olimpiadas lingüísticas para LLMs.

Benchmark	Año	Tamaño	Rasgo principal
<i>PuzzLing Machines</i>	2020	~100 problemas	Antecedente centrado en problemas de tipo piedra Rosetta.
<i>modeLing</i>	2024	48 problemas, 272 preguntas	Problemas originales diseñados para reducir el riesgo de contaminación de los datos de entrenamiento.
<i>Linguini</i>	2024	160 problemas, 894 preguntas	Benchmark de referencia utilizado en los experimentos de este trabajo.
<i>LingOly</i>	2024	1.133 problemas	Conjunto de mayor tamaño, con distintos formatos y niveles de dificultad.

2.4. Estrategias de prompting para razonamiento lingüístico

Ante los pobres resultados obtenidos con prompting directo, varios trabajos han explorado estrategias más sofisticadas para mejorar la capacidad de los LLMs en esta tarea.

2.4.1. Aprendizaje en contexto con ejemplos

El aprendizaje en contexto (in-context learning, ICL) es la capacidad de un modelo de lenguaje para realizar una tarea nueva a partir de la información incluida directamente en el prompt, sin necesidad de actualizar sus parámetros.

En los problemas de olimpiadas lingüísticas, esta capacidad resulta fundamental. El modelo recibe un conjunto reducido de pares formados por una expresión en una lengua desconocida y su traducción. A partir de estos ejemplos debe deducir vocabulario, identificar regularidades morfológicas y aplicar las reglas extraídas a nuevas frases.

Conviene diferenciar este mecanismo del few-shot prompting. En una configuración few-shot, el modelo recibe ejemplos adicionales que muestran cómo debe resolverse la tarea o cómo debe estructurarse la respuesta. Estos ejemplos no forman parte necesariamente del problema lingüístico concreto, sino que actúan como demostraciones previas.

Sánchez y cols. (2024) comparan distintas configuraciones con un número variable de demostraciones y observan resultados heterogéneos. Algunos modelos mejoran al recibir ejemplos adicionales, mientras que otros empeoran. Una posible explicación es que las demostra-

ciones introducen información útil sobre el formato de la tarea, pero también pueden añadir ruido cuando proceden de lenguas o estructuras poco relacionadas con el problema que se desea resolver.

Esta observación resulta relevante para el presente trabajo: proporcionar más información no garantiza por sí solo una mejora del rendimiento. También es necesario estudiar cómo se presenta esa información y en qué orden se incorpora al razonamiento del modelo.

2.4.2. Razonamiento paso a paso

Zhu y cols. (2025) proponen una estrategia de descomposición paso a paso inspirada en el chain-of-thought prompting (Wei y cols., 2022), que en resumen, busca dividir un problema complejo en pasos intermedios más manejables.

En el contexto de las olimpiadas lingüísticas, el razonamiento paso a paso puede consistir en identificar primero el vocabulario básico, después analizar los patrones morfológicos, formular reglas gramaticales y, finalmente, aplicar esas reglas a las frases de prueba.

En el estudio de la estrategia de la descomposición progresiva, en lugar de presentar todos los ejemplos de entrenamiento de golpe, organizan la información en etapas lingüísticas sucesivas: primero se presenta el léxico y el orden de palabras básico, después los patrones fonológicos, luego la morfosintaxis y finalmente la sintaxis compleja. En cada etapa, el modelo actualiza su comprensión de la lengua antes de pasar a la siguiente. Este enfoque obtiene mejoras significativas respecto al prompting directo en lenguas con morfología verbal compleja.

2.4.3. Prompting analógico

Ramji y Ramji (2024) proponen una estrategia basada en el razonamiento analógico. En lugar de depender únicamente de los ejemplos incluidos en el problema, el modelo genera o recupera ejemplos auxiliares relacionados con fenómenos lingüísticos similares.

La idea consiste en aprovechar el conocimiento general adquirido durante el preentrenamiento. Por ejemplo, si una lengua desconocida presenta un patrón de concordancia o una estructura sintáctica poco habitual, el modelo puede intentar relacionarlo con estructuras semejantes presentes en otras lenguas que conoce mejor.

Esta estrategia puede facilitar la identificación de determinados patrones, pero también introduce una limitación importante: el razonamiento deja de depender exclusivamente de la información proporcionada en el enunciado. El modelo puede apoyarse en analogías incorrectas o trasladar reglas de una lengua a otra sin suficiente evidencia.

En este trabajo no se utiliza prompting analógico, ya que uno de los objetivos es evaluar hasta qué punto el modelo puede resolver los problemas a partir de los ejemplos disponibles, sin recibir pistas sobre la lengua ni sobre su familia lingüística. No obstante, esta aproximación resulta relevante como posible línea de ampliación futura.

2.4.4. Gramática en contexto

Tanzer y cols. (2024) estudian otra aproximación: proporcionar al modelo una gramática descriptiva de la lengua como parte del contexto. Este enfoque, denominado one-book promp-

ting, permite trabajar con lenguas para las que existe documentación lingüística, aunque no se disponga de grandes corpus paralelos.

A diferencia de los problemas de olimpiadas lingüísticas, el modelo no debe deducir todas las reglas exclusivamente a partir de ejemplos. En su lugar, puede consultar una descripción explícita de la lengua y utilizarla para traducir nuevas frases.

Los resultados muestran que incluir una gramática puede mejorar el rendimiento del modelo, especialmente cuando se combina con ejemplos. Esta estrategia resulta prometedora para aplicaciones reales de traducción y documentación lingüística, ya que muchas lenguas con pocos recursos digitales sí disponen de gramáticas, diccionarios o estudios elaborados por especialistas.

Sin embargo, este enfoque responde a un objetivo diferente del abordado en el presente trabajo. Aquí se pretende evaluar la capacidad de deducción a partir de un conjunto reducido de pares de traducción, sin incorporar documentación externa sobre la lengua.

2.5. Análisis de las dificultades de los LLMs en puzzles lingüísticos

Más allá de cuantificar el rendimiento de los modelos, algunos trabajos recientes han intentado explicar por qué los LLMs fallan en estos puzzles.

Choudhary y cols. (2025) realizan el análisis más sistemático hasta la fecha, anotando 629 problemas en 41 lenguas de baja densidad con 50 características lingüísticas y correlacionando estas características con el rendimiento de los modelos. Sus conclusiones principales son dos. En primer lugar, la complejidad morfológica es el factor predictivo más importante del fracaso: cuanto más compleja es la morfología de la lengua (más morfemas por palabra, más procesos de afijación y fusión), peor rinden los modelos. En segundo lugar, los modelos rinden significativamente mejor en puzzles cuyas características lingüísticas también están presentes en inglés, lo que sugiere que el razonamiento metalingüístico de los LLMs está sesgado hacia los patrones del inglés y las lenguas de alta densidad de recursos.

Este hallazgo tiene implicaciones directas para la interpretación de los resultados de este trabajo: la variación en el rendimiento entre los distintos puzzles evaluados puede explicarse en parte por las diferencias tipológicas entre las lenguas seleccionadas.

El trabajo más reciente del campo, Majmudar y Filatova (2025), extiende la evaluación a modelos de razonamiento como o1 de OpenAI y encuentra que estos modelos superan a los humanos en la mayoría de categorías de puzzles, con la excepción notable de los sistemas de escritura no familiares y las lenguas más alejadas tipológicamente del inglés. Este resultado abre una nueva perspectiva sobre el papel del razonamiento explícito en la resolución de puzzles lingüísticos, y sugiere que los modelos con capacidad de razonamiento extendido representan el siguiente paso en esta línea de investigación.

2.6. Síntesis y posicionamiento del trabajo

La revisión de la literatura permite extraer varias conclusiones. En primer lugar, los modelos de lenguaje de gran escala son capaces de identificar algunos patrones lingüísticos a partir de pocos ejemplos, pero su rendimiento continúa siendo limitado en comparación con el humano en numerosos tipos de problemas. En segundo lugar, la dificultad no depende únicamente del número de ejemplos disponibles: factores como la complejidad morfológica, la escasa

representación digital de determinadas lenguas y la segmentación interna de las palabras influyen de forma importante en los resultados. Finalmente, los trabajos revisados muestran que la forma de presentar la información al modelo puede modificar considerablemente su rendimiento.

A partir de estas observaciones, este Trabajo de Fin de Grado estudia distintas estrategias de prompting orientadas a mejorar la resolución de problemas de olimpiadas lingüísticas. El objetivo no consiste únicamente en medir cuántas traducciones correctas genera cada modelo, sino también en analizar cómo construye sus hipótesis, hasta qué punto es capaz de revisarlas y en qué momento aparecen los errores.

La primera aportación del trabajo es una estrategia incremental por combinaciones crecientes de ejemplos. En lugar de presentar todos los pares de entrenamiento en un único bloque, el modelo recibe subconjuntos de tamaño progresivamente mayor. Tras analizar cada subconjunto, debe actualizar un diccionario y un conjunto de reglas gramaticales. Estas hipótesis no se consideran definitivas: el modelo puede ampliarlas, corregirlas o eliminar aquellas que resulten incompatibles con la nueva evidencia.

Esta estrategia se inspira en el proceso que sigue una persona al enfrentarse a una lengua desconocida. Habitualmente, el razonamiento humano no consiste en formular de inmediato una gramática completa, sino en comparar ejemplos, detectar elementos recurrentes, proponer interpretaciones provisionales y revisarlas cuando aparecen contradicciones. La incorporación explícita de esta lógica de revisión pretende evitar que el modelo mantenga hipótesis iniciales incorrectas durante todo el proceso de resolución.

La segunda aportación es la evaluación de un baseline con una fase adicional de revisión. En esta variante, el modelo recibe todos los ejemplos del problema desde el inicio, como en el baseline directo, pero debe revisar su primera propuesta antes de generar la respuesta definitiva. El objetivo es comprobar si una segunda lectura de los mismos datos permite detectar errores, corregir inconsistencias y mejorar las traducciones sin necesidad de introducir información adicional.

Esta comparación permite distinguir dos posibles fuentes de mejora. Por una parte, la presentación progresiva de los ejemplos puede facilitar la construcción del conocimiento lingüístico. Por otra, la simple obligación de revisar una respuesta inicial puede mejorar el resultado incluso cuando el contexto permanece constante. Separar ambos efectos resulta necesario para evaluar con mayor precisión la utilidad real de la estrategia incremental.

La tercera aportación es la incorporación de pautas inspiradas en la resolución humana de los problemas. El modelo no recibe únicamente la orden de traducir, sino que debe identificar vocabulario, analizar regularidades morfológicas, formular reglas, señalar contradicciones y justificar cualquier modificación de sus hipótesis. De este modo, el prompt intenta reproducir algunos de los pasos observados durante la resolución humana, sin asumir que el proceso interno del modelo sea equivalente al de una persona.

En los trabajos revisados se han estudiado estrategias como el aprendizaje en contexto, el razonamiento paso a paso, el prompting analógico y la incorporación de gramáticas descriptivas. Sin embargo, dentro de la literatura analizada, la combinación sistemática de revisión explícita, presentación incremental por combinaciones crecientes y comparación con un baseline de doble revisión no ha recibido la misma atención.

Este trabajo se sitúa, por tanto, en un punto intermedio entre la evaluación cuantitativa y el análisis cualitativo. Por un lado, compara el rendimiento de distintos modelos y estrategias

mediante métricas automáticas. Por otro, estudia cómo evolucionan el diccionario y las reglas formuladas por el modelo, y contrasta ese proceso con las estrategias empleadas durante la resolución humana de un problema representativo.

La finalidad no es demostrar que los modelos razonan de la misma manera que una persona, sino analizar hasta qué punto una organización más estructurada del proceso de resolución permite obtener respuestas más correctas, coherentes y revisables.

3. Objetivos

Este capítulo presenta el objetivo principal del trabajo y los objetivos específicos que se derivan de él, así como las restricciones y condicionantes que han guiado el desarrollo del proyecto.

3.1. Objetivo principal

El objetivo principal de este Trabajo de Fin de Grado es diseñar, implementar y evaluar estrategias de *prompting* para que modelos de lenguaje de gran escala puedan resolver puzzles de olimpiadas lingüísticas en idiomas que nunca han formado parte de su entrenamiento, apoyándose únicamente en razonamiento metalingüístico y sin recibir ninguna información previa sobre la lengua en cuestión.

3.2. Objetivos específicos

De este objetivo principal se derivan los siguientes objetivos específicos:

1. **Revisar el estado del arte** en razonamiento lingüístico con modelos de lenguaje de gran escala, con especial atención a los benchmarks derivados de la Olimpiada Internacional de Lingüística (LINGUINI, LingOly, modeLing) y a las estrategias de prompting más relevantes de la literatura reciente.
2. **Diseñar e implementar cinco estrategias de prompting** diferenciadas y comparables entre sí:
 - Una estrategia de *verificación* (M0) que comprueba si el modelo conoce el idioma antes del experimento, garantizando la validez del control.
 - Una estrategia *baseline* (M1) que sirve como punto de referencia directo con la literatura existente, presentando todos los ejemplos de entrenamiento de forma simultánea.
 - Una estrategia de presentación *incremental por combinaciones crecientes* (M2) de ejemplos, con construcción explícita y revisable de un diccionario y un conjunto de reglas gramaticales.
 - Una estrategia *paso a paso* (M3) ordenada por nivel lingüístico (léxico, fonología, morfosintaxis y sintaxis), inspirada en el trabajo de Zhu y cols. (2025).
 - Una estrategia de *inspiración humana* (M4) que proporciona al modelo el razonamiento explícito de un experto humano resolviendo un puzzle diferente, para guiar el proceso inductivo.

- Una estrategia de *autocorrección iterativa* (M5), en la que el modelo revisa sus propias hipótesis en un segundo turno buscando contradicciones internas, inspirada en el paradigma *Self-Refine* (?).
3. **Construir un sistema experimental reproducible** que automatice la ejecución de los experimentos, el registro de las respuestas del modelo y el cálculo de métricas, permitiendo auditar cualquier resultado de forma independiente.
 4. **Evaluar las estrategias** sobre puzzles reales del benchmark LINGUINI en lenguas de diversas familias lingüísticas, en ambas direcciones de traducción (lengua desconocida → inglés e inglés → lengua desconocida), utilizando modelos de tres tamaños distintos.
 5. **Comparar los resultados obtenidos** con los valores de referencia publicados en Sánchez y cols. (2024) mediante las métricas BLEU, chrF++ y TER, e identificar en qué condiciones cada estrategia supera o iguala el estado del arte.
 6. **Realizar un análisis cualitativo** comparando el proceso de resolución humana de los puzzles con el razonamiento explícito generado por los modelos, identificando los puntos de divergencia y los patrones de error más frecuentes según la familia lingüística, el tamaño del modelo y la dirección de traducción.

3.3. Restricciones y condicionantes

El desarrollo del trabajo está sujeto a las siguientes restricciones:

Acceso a los modelos: se utilizan exclusivamente modelos accesibles mediante APIs gratuitas (Groq, OpenRouter y Cerebras), lo que limita los modelos disponibles pero garantiza la reproducibilidad del trabajo sin coste computacional.

Límites de tasa: las APIs gratuitas imponen límites de peticiones por minuto y por día, lo que condiciona el número de experimentos que pueden ejecutarse en una sesión y obliga a introducir retardos entre llamadas. Este condicionante es especialmente notable en M2, cuya naturaleza iterativa genera hasta nueve llamadas consecutivas por puzzle.

Datos: los puzzles utilizados provienen exclusivamente del benchmark LINGUINI (Sánchez y cols., 2024), cuya descarga requiere aceptar los términos de uso del corpus de la IOL (licencia CC-BY-SA 4.0). No se utilizan puzzles de elaboración propia para mantener la comparabilidad con la literatura.

Ausencia de reentrenamiento: no se realiza ningún tipo de *fine-tuning* ni adaptación de los modelos. Todas las estrategias operan en modo *zero-shot* o *few-shot* mediante prompting, sin modificar los pesos del modelo.

Determinismo de las respuestas: todos los experimentos se ejecutan con temperatura cero (*temperature=0*), de modo que las respuestas del modelo son deterministas y reproducibles para una misma versión del modelo y del prompt.

4. Metodología y datos

Este capítulo describe el diseño experimental del trabajo: el dataset utilizado, los modelos evaluados, las estrategias de prompting implementadas, el sistema de evaluación automática y el protocolo de análisis cualitativo.

4.1. Dataset: Linguini

Los experimentos utilizan puzzles extraídos del benchmark LINGUINI (Sánchez y cols., 2024), que provienen del corpus de la Olimpiada Internacional de Lingüística (IOL). Se seleccionaron exclusivamente puzzles de tipo *sequence transduction* (traducción), que es la categoría más adecuada para evaluar razonamiento metalingüístico inductivo y la única que permite comparación directa con los trabajos de referencia (Sánchez y cols., 2024; Zhu y cols., 2025).

4.1.1. Criterios de selección

Los criterios de selección de los puzzles fueron los siguientes: (1) pertenecer a familias lingüísticas distintas entre sí, para maximizar la diversidad tipológica; (2) disponer de al menos ocho pares de entrenamiento, para que la estrategia incremental tenga margen suficiente para construir conocimiento progresivo; (3) disponer de al menos tres pares de test, para que las métricas automáticas sean estadísticamente significativas; y (4) que el idioma en cuestión fuera desconocido para los modelos evaluados, criterio verificado mediante la estrategia M0 (véase la Sección 4.3.2).

Se seleccionaron puzzles en ambas direcciones de traducción: de la lengua desconocida al inglés (*lang2en*) y del inglés a la lengua desconocida (*en2lang*). Esta doble dirección permite analizar si las estrategias de prompting son igualmente eficaces cuando el modelo debe producir morfología de una lengua que no conoce, en lugar de interpretar significado.

4.1.2. Puzzles seleccionados

El conjunto final comprende 16 puzzles en 11 lenguas y 10 familias lingüísticas distintas, cubriendo regiones geográficas de Asia, Oceanía, América y África. La Tabla 4.1 recoge las características de cada puzzle.

Cinco lenguas están representadas en ambas direcciones (Hakhun, N|uuki, Yonggom, Wambaya y Apurinã), lo que permite comparar directamente el rendimiento de cada estrategia en función de la dirección de traducción. Las once lenguas restantes, con sus correspondientes familias, permiten analizar si los patrones observados dependen de la tipología lingüística o son generalizables.

Tabla 4.1: Puzzles utilizados en los experimentos. Dir.: dirección de traducción ($\rightarrow$ EN = lengua desconocida al inglés; EN $\rightarrow$ = inglés a lengua desconocida).

ID	Lengua	Familia	Región	Dir.	Train	Test
012018020100	Hakhun	Sino-Tibetana	India	$\rightarrow$ EN	10	6
012018020200	Hakhun	Sino-Tibetana	India	EN $\rightarrow$	10	4
012022030100	N uuki	Tuu	Sudáfrica	$\rightarrow$ EN	16	6
012022030200	N uuki	Tuu	Sudáfrica	EN $\rightarrow$	16	6
012019010100	Yonggom	Trans-New Guinea	Papua-NG	$\rightarrow$ EN	11	6
012019010200	Yonggom	Trans-New Guinea	Papua-NG	EN $\rightarrow$	11	4
012015040100	Wambaya	Mirndi	Australia	$\rightarrow$ EN	17	6
012015040200	Wambaya	Mirndi	Australia	EN $\rightarrow$	17	5
012023020100	Apurinã	Arawakan	Brasil	$\rightarrow$ EN	16	3
012023020300	Apurinã	Arawakan	Brasil	EN $\rightarrow$	16	6
012023030100	Coastal Marind	Trans-New Guinea	Indonesia	$\rightarrow$ EN	17	5
012012010200	Dyirbal	Pama-Nyungan	Australia	$\rightarrow$ EN	16	3
012016030100	Kunuz Nubian	Nilo-Saharan	Egipto/Sudán	$\rightarrow$ EN	10	5
012008050100	Inuktitut	Eskimo-Aleut	Canadá	$\rightarrow$ EN	12	6
012005010100	Tzotzil	Maya	México	$\rightarrow$ EN	15	4
012006010100	Lakhota	Sioux	EE.UU.	$\rightarrow$ EN	13	4

4.1.3. Formato de los datos

Los puzzles de LINGUINI se distribuyen en formato JSONL. Para este trabajo se implementó un conversor que transforma cada problema al formato JSON del sistema experimental, con los campos `id`, `language`, `language_family`, `translation_direction`, `train_pairs` y `test_pairs`. Este formato estandarizado permite añadir nuevos puzzles de forma sencilla y garantiza la reproducibilidad de los experimentos.

4.2. Modelos evaluados

Se evaluaron tres modelos de lenguaje de gran escala de distintos tamaños, todos accesibles mediante APIs gratuitas, lo que garantiza la reproducibilidad del trabajo. La Tabla 4.2 recoge sus características principales.

Tabla 4.2: Modelos evaluados

Denominación	Modelo	Organización	Parámetros	Tier	API
Llama-8B	Llama 3.1 8B Instant	Meta	8B	Débil	Groq
Llama-70B	Llama 3.3 70B Versatile	Meta	70B	Medio	Groq
GPT-OSS-120B	GPT-OSS 120B	OpenAI	120B	Fuerte	OpenRouter

Los tres modelos son transformers *decoder-only*, por lo que la diferencia principal entre ellos es el tamaño (número de parámetros) y el proceso de preentrenamiento. La elección de dos modelos de la familia Llama en tamaños diferentes (8B y 70B) permite aislar el efecto

del tamaño manteniendo constante la arquitectura base. El tercer modelo (GPT-OSS-120B) pertenece a una organización distinta y tiene un proceso de preentrenamiento diferente, lo que añade diversidad a la comparación y permite evaluar si los patrones observados son específicos de los modelos Llama o generalizables.

Todos los modelos se emplean con temperatura cero (`temperature=0`) para garantizar la reproducibilidad de los resultados, y sin ejemplos adicionales del formato de respuesta esperado más allá de los propios del puzzle.

4.3. Estrategias de prompting

Se diseñaron e implementaron seis estrategias de prompting, identificadas como M0 a M5. Las cinco estrategias de traducción (M1 a M5) comparten una instrucción metalingüística base.

4.3.1. Instrucción metalingüística base

Las estrategias M1 a M5 parten de una instrucción común que establece el marco de razonamiento del modelo: actuar como analista lingüístico que nunca ha visto la lengua objetivo, razonar exclusivamente a partir de los ejemplos proporcionados, y tratar todas las hipótesis como provisionales y revisables. Específicamente, se le indica que no utilice conocimiento previo sobre la lengua o su familia, que construya explícitamente un diccionario y reglas gramaticales, y que muestre su razonamiento antes de llegar a ninguna conclusión. Este enfoque se inspira en el prompting metalingüístico de Zhu y cols. (2025) y en los principios del *chain-of-thought* (Wei y cols., 2022).

4.3.2. M0: Verificación

Antes de ejecutar las estrategias de traducción, M0 comprueba que el modelo efectivamente desconoce el idioma del puzzle. Se le presenta una única frase del conjunto de test sin ningún contexto y se le pide que intente traducirla. Si el modelo produce una traducción con chrF++ superior a 20, se considera que posiblemente conoce el idioma y el experimento se marca como potencialmente inválido. Esta estrategia de control es necesaria porque los modelos más grandes pueden tener acceso a datos de lenguas que se consideraban extremadamente minoritarias.

4.3.3. M1: Baseline

La estrategia baseline proporciona al modelo todos los pares de entrenamiento del puzzle en un único prompt y le pide que genere directamente el diccionario, las reglas gramaticales y las traducciones de las frases de test. Esta estrategia replica el prompting directo utilizado como referencia en Sánchez y cols. (2024) y sirve como punto de comparación para las demás estrategias.

4.3.4. M2: Incremental por combinaciones

La estrategia incremental constituye la aportación metodológica principal de este trabajo. En lugar de presentar todos los ejemplos de golpe, se presentan subconjuntos de tamaño creciente: primero las primeras 2 frases, luego las primeras 3, y así sucesivamente hasta presentar las n frases de entrenamiento, para un total de $\min(n - 1, 15)$ pasos.

En cada paso, el modelo recibe el subconjunto de frases junto con su hipótesis acumulada del paso anterior (diccionario y reglas) y debe realizar cuatro operaciones en orden:

1. **Análisis** (STEP A): razonar explícitamente sobre los nuevos ejemplos antes de actualizar ninguna hipótesis, identificando morfemas y detectando contradicciones con el conocimiento previo.
2. **Diccionario revisado** (STEP B): actualizar el diccionario añadiendo entradas nuevas, corrigiendo entradas imprecisas y eliminando entradas refutadas (marcadas como [DELETED: razón]).
3. **Reglas revisadas** (STEP C): actualizar el conjunto de reglas gramaticales con la misma lógica.
4. **Traducciones** (STEP D): traducir las frases de test usando el diccionario y las reglas actualizados.

La posibilidad explícita de eliminar hipótesis erróneas es un elemento diferencial respecto a las estrategias de la literatura, donde el conocimiento acumulado solo crece. Se fundamenta en la observación de que los modelos tienden a mantener hipótesis incorrectas incluso con evidencia contradictoria, si el prompt no les obliga explícitamente a revisarlas (Zhu y cols., 2025).

Para la comparación entre metodologías se reporta el mejor paso de M2 según chrF++. La evolución completa paso a paso se conserva en los logs para el análisis de estabilidad del aprendizaje incremental.

4.3.5. M3: Step-by-step lingüístico

La estrategia step-by-step está inspirada en Zhu y cols. (2025) y organiza la presentación de los ejemplos según una jerarquía lingüística de cuatro etapas:

1. **Léxico** (35 % de los ejemplos): frases simples para identificar vocabulario básico y orden de palabras fundamental.
2. **Fonología** (15 % de los ejemplos): frases que exhiben variaciones fonológicas (alternancias vocálicas, alófonos, asimilaciones).
3. **Morfosintaxis** (30 % de los ejemplos): frases con concordancia de persona, número, tiempo y aspecto.
4. **Sintaxis** (20 % de los ejemplos): frases con estructuras complejas (negación, interrogación, subordinación).

Esta partición es heurística cuando los puzzles carecen de anotación lingüística explícita, como ocurre en LINGUINI. Al igual que en M2, el modelo acumula diccionario y reglas entre etapas y puede corregir hipótesis previas.

4.3.6. M4: Inspiración humana

La estrategia de inspiración humana proporciona al modelo el razonamiento explícito de un humano resolviendo un puzzle diferente, para guiar el proceso inductivo. El razonamiento utilizado como ejemplo es la resolución detallada del puzzle de Hakhun (véase la Sección 4.6), que incluye los patrones identificados en cada frase, las hipótesis provisionales formuladas y cómo se corrigen o confirman.

El modelo recibe este razonamiento como referencia de proceso, no de contenido: se le indica explícitamente que no transfiera ni vocabulario ni reglas del ejemplo, ya que pertenecen a una lengua completamente distinta. El objetivo es que el modelo imite la estrategia de análisis (identificar invariantes, aislar morfemas, revisar hipótesis progresivamente) aplicándola a la nueva lengua.

Para garantizar la validez de los resultados, M4 no se ejecuta sobre el puzzle de Hakhun, dado que es el mismo puzzle cuyo razonamiento se proporciona como ejemplo.

4.3.7. M5: Autocorrección iterativa

La estrategia de autocorrección iterativa está inspirada en el paradigma *Self-Refine* (?) y opera en dos turnos. En el primer turno, el modelo realiza una traducción completa siguiendo el mismo prompt que M1. En el segundo turno, recibe sus propias traducciones y el diccionario y reglas que infirió, y se le pide que identifique contradicciones internas explícitas antes de producir una versión corregida. La instrucción es deliberadamente precisa: debe citar la regla o entrada de diccionario que cada traducción viola o que no está cubierta, y solo entonces producir las traducciones finales.

Esta estrategia evalúa si el razonamiento metalingüístico del propio modelo es suficiente para detectar y corregir errores de coherencia sin recibir información adicional sobre la lengua.

4.4. Sistema experimental

Se implementó un sistema experimental en Python (`experiment_runner.py`) que automatiza la ejecución de los experimentos, el cálculo de métricas y el registro de resultados. Dado un puzzle y un modelo, el sistema ejecuta las seis estrategias en orden, calcula las métricas en cada paso y guarda un log completo en formato JSON con las hipótesis extraídas, las métricas calculadas, el diccionario y las reglas acumulados en cada paso, y opcionalmente el prompt y la respuesta completa del modelo.

4.4.1. Extracción de traducciones.

Un aspecto técnico relevante es la extracción de las traducciones de la respuesta libre del modelo. Los modelos no siempre siguen el formato solicitado: algunos utilizan comillas tipográficas (""), cursivas en Markdown (**texto**), flechas unicode (→) u otros formatos. Se implementó un extractor multicapa con expresiones regulares que cubre los patrones de

salida observados empíricamente, con filtros explícitos para descartar entradas de diccionario, instrucciones de revisión y meta-análisis lingüístico que los modelos a veces incluyen en la sección de traducciones. Este extractor fue validado de forma independiente comparando las métricas calculadas contra un recálculo manual sobre los mismos datos.

4.4.2. Parámetros de ejecución.

Las llamadas a los modelos se realizan con temperatura cero y un límite de 4096 tokens de salida. Para respetar los límites de tasa de las APIs gratuitas, se introduce un retardo de 10 segundos entre los pasos consecutivos de M2 y de 20 segundos entre los dos turnos de M5. El tiempo de espera máximo por llamada es de 150 segundos, configurado especialmente para OpenRouter, cuya latencia con el modelo de 120B parámetros es significativamente mayor.

4.5. Métricas de evaluación

Se utilizan cuatro métricas automáticas de evaluación, calculadas con la biblioteca **sacrebleu** (Post, 2018):

BLEU (Papineni y cols., 2002) calcula la precisión de n-gramas compartidos entre la hipótesis del modelo y la referencia, con una penalización por brevedad. Su principal limitación en este contexto es que requiere coincidencia exacta de palabras, lo que la hace excesivamente estricta para lenguas con morfología compleja. Se incluye por compatibilidad con la literatura existente ($\uparrow$ mejor, rango 0–100).

chrF++ (Popović, 2017) es la métrica principal de este trabajo. Opera a nivel de n-gramas de caracteres y de palabras (**word_order=2**), lo que la hace más tolerante con variaciones morfológicas y más adecuada para lenguas aglutinantes. Es la métrica secundaria del propio benchmark LINGUINI (Sánchez y cols., 2024) y la utilizada para comparar los mejores pasos de M2 entre sí ($\uparrow$ mejor, rango 0–100).

TER (?) (*Translation Edit Rate*) mide el número mínimo de ediciones por palabra necesarias para transformar la hipótesis en la referencia. A diferencia de BLEU y chrF++, es una métrica de error: un valor de 0 indica traducción perfecta, y valores superiores a 100 indican que se necesitan más ediciones que palabras hay en la referencia ($\downarrow$ mejor).

Exact Match (EM) cuenta el porcentaje de traducciones exactamente correctas, ignorando mayúsculas y puntuación. Con pocos pares de test por puzzle (entre 3 y 6), cada acierto representa entre un 17 y un 33 % del score, lo que hace esta métrica discreta pero muy directamente interpretable ($\uparrow$ mejor, rango 0–100 %).

4.6. Análisis cualitativo

Además de la evaluación automática, se realizó un análisis cualitativo comparando el proceso de resolución humana con el del modelo en el puzzle de Hakhun (Sino-Tibetana, India; 10 frases de entrenamiento, 6 de test). El objetivo es identificar en qué aspectos del razonamiento metalingüístico los modelos divergen de los humanos, más allá de lo que capturan las métricas automáticas.

4.6.1. Protocolo de resolución humana

El protocolo consistió en resolver el puzzle anotando explícitamente cada paso del razonamiento: qué patrones se identificaron primero, qué hipótesis se formularon, cuándo y por qué se corrigieron, y cómo se aplicaron a las frases de test.

4.6.2. Proceso de resolución observado

El proceso humano siguió el siguiente orden: primero se identificaron los elementos invariantes (**ne** como marcador de pregunta, presente en todas las frases), luego los pronombres de sujeto (**ŋa**=I, **n**=you(sg), **nirum**=we, **tarum**=they), después los verbos (**lapk i**=see, **c am**=know, **lan**=beat, **ka**=go, **ip**=sleep), y finalmente el sistema aspectual: las terminaciones con **o t** - marcan pasado, las demás marcan presente. El marcador **kəmə** se identificó como marcador de transitividad sin traducción directa al inglés. Con este protocolo se obtuvieron las seis traducciones de test correctas.

4.6.3. Comparación con el modelo

La comparación con la respuesta del modelo en M1 revela un patrón consistente: el modelo identifica correctamente el léxico básico (verbos y pronombres de sujeto) pero falla sistemáticamente en el sistema aspectual (pasado vs. presente). El modelo menciona en su razonamiento la posibilidad de que **o t** marque pasado, pero no lo aplica de forma consistente a todas las formas verbales. Este resultado está en línea con el hallazgo de Zhu y cols. (2025) de que los modelos rinden peor en puzzles con mayor complejidad morfológica, ya que la concordancia de tiempo, aspecto y objeto en Hakhun es precisamente el elemento más complejo del puzzle.

En M2, el modelo con subconjuntos pequeños produce traducciones degradadas por insuficiencia de evidencia para deducir el sistema pronominal de objeto. El rendimiento mejora en pasos intermedios pero puede decaer en los pasos finales en los modelos más débiles, posiblemente por sobrecarga del contexto del prompt. El humano mostró el patrón opuesto: con solo dos frases ya identificó los elementos invariantes, y cada par adicional refinó hipótesis previas sin degradar las ya establecidas.

Este contraste sugiere que la ventaja de los modelos grandes en tareas de traducción metalingüística no proviene de un proceso de razonamiento más eficiente, sino de una mayor capacidad para mantener coherencia en contextos largos y de una base de conocimiento tipológico más amplia sobre la que apoyar las inferencias.

5. Desarrollo del sistema

Este capítulo describe en detalle la implementación del sistema experimental: su arquitectura general, el diseño de los prompts de cada estrategia, el extractor de traducciones, el proceso de construcción del dataset y los problemas técnicos encontrados durante el desarrollo con sus soluciones.

5.1. Arquitectura general

El sistema experimental se implementó en Python y se organizó en torno a un script principal denominado `experiment_runner.py`. Su función es automatizar el proceso completo de evaluación: cargar un puzzle, seleccionar un modelo, ejecutar las estrategias de prompting, extraer las traducciones generadas, calcular las métricas y almacenar los resultados en un archivo JSON.

El sistema sigue el mismo flujo general para todos los experimentos:

1. Carga del puzzle desde un archivo JSON.
2. Selección del modelo y de las estrategias que se desean evaluar.
3. Construcción del prompt correspondiente a cada estrategia.
4. Envío de la petición al proveedor mediante su API.
5. Recepción de la respuesta textual generada por el modelo.
6. Extracción automática de las traducciones.
7. Comparación con las referencias correctas y cálculo de métricas.
8. Almacenamiento de los resultados y de los pasos intermedios en un log JSON.

La Figura 5.1 resume este proceso.

La coordinación de las estrategias se realiza mediante una función principal que recorre las metodologías solicitadas y almacena sus resultados de manera independiente. El Código 5.1 muestra una versión simplificada de esta lógica.

Código 5.1: Flujo principal de ejecución de un experimento

```
1 def run_experiment(puzzle, model_key, api_keys, strategies):
2     log = {
3         "metadata": build_metadata(puzzle, model_key, strategies),
4         "puzzle": puzzle,
5         "results": {}
6     }
7
```

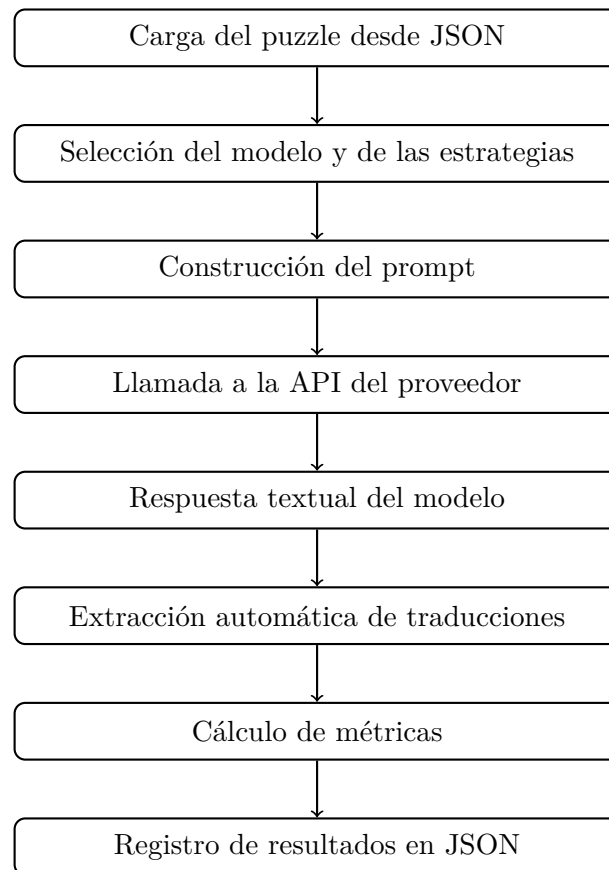


Figura 5.1: Flujo general del sistema experimental.

```

8 for strategy_id in strategies:
9     strategy_name = STRATEGY_NAMES[strategy_id]
10
11     try:
12         result = RUNNERS[strategy_id](
13             puzzle,
14             model_key,
15             api_keys
16         )
17         log["results"][strategy_name] = result
18
19     except Exception as error:
20         log["results"][strategy_name] = {
21             "error": str(error)
22         }
23
24     log["summary"] = build_summary(log["results"])
25     save_log(log)
26
27     return log

```

Cada estrategia se ejecuta dentro de un bloque de control de excepciones. Esta decisión permite que un fallo puntual no detenga necesariamente el experimento completo. Por ejemplo, una petición puede superar el tiempo máximo de espera o una API puede rechazar

temporalmente una llamada por haber alcanzado su límite de uso. En ese caso, el error queda registrado en el log y el sistema puede continuar con las estrategias restantes.

5.1.1. Formato de entrada

Cada puzzle se almacena en un archivo JSON independiente. El formato incluye información descriptiva sobre la lengua y dos listas claramente diferenciadas: **train_pairs** y **test_pairs**.

Los pares de entrenamiento se incorporan al prompt y constituyen la única fuente de información lingüística disponible para el modelo durante la resolución. Los pares de test se reservan para la evaluación automática. Sus traducciones correctas se utilizan únicamente después de recibir la respuesta del modelo, por lo que no aparecen en el prompt.

El Código 5.2 muestra una versión reducida del puzzle de Hakhun utilizado durante el desarrollo.

Código 5.2: Estructura simplificada de un puzzle en formato JSON

```
1 {  
2   "id": "linguini_012018020100",  
3   "language": "Hakhun",  
4   "language_family": "Sino-Tibetan",  
5   "translation_direction": "lang2en",  
6  
7   "train_pairs": [  
8     {  
9       "source": "ḡa ka k ne",  
10      "target": "Do I go?"  
11    },  
12    {  
13      "source": "n ip tu ne",  
14      "target": "Did you(sg) sleep?"  
15    }  
16  ],  
17  
18  "test_pairs": [  
19    {  
20      "source": "n ip ku ne",  
21      "target": "Do you(sg) sleep?"  
22    }  
23  ]  
24 }
```

El campo **translation_direction** permite distinguir entre los puzzles en los que el modelo traduce desde la lengua desconocida al inglés (**lang2en**) y aquellos en los que debe generar expresiones en la lengua desconocida a partir de frases en inglés (**en2lang**).

5.1.2. Formato del log JSON

Cada combinación de puzzle y modelo genera un archivo JSON de resultados. El objetivo de este registro es permitir tanto la comparación cuantitativa entre estrategias como la revisión manual de casos concretos.

El log contiene tres bloques principales:

1. **metadata**: información general del experimento, como el modelo, la lengua, la dirección de traducción y las estrategias ejecutadas.

2. **puzzle**: copia de los pares de entrenamiento y de test utilizados.
3. **results**: resultados obtenidos por cada estrategia, incluyendo traducciones extraídas, métricas, latencias y pasos intermedios.

Las estrategias iterativas, como M2 y M3, almacenan además la evolución de las hipótesis. En cada paso se registran el número de ejemplos utilizados, el diccionario acumulado, las reglas gramaticales inferidas y las métricas obtenidas. Esto permite estudiar si el rendimiento mejora progresivamente o si una hipótesis incorrecta provoca una degradación posterior.

El Código 5.3 muestra una versión simplificada de la estructura de salida.

Código 5.3: Estructura simplificada del log de resultados

```

1 {
2   "metadata": {
3     "puzzle_id": "linguini_012018020100",
4     "language": "Hakhun",
5     "model_key": "llama-3.3-70b-groq",
6     "translation_direction": "lang2en"
7   },
8
9   "results": {
10    "baseline": {
11      "best_metrics": {
12        "corpus_chrfpp": 54.85,
13        "exact_match_corpus": 16.67
14      }
15    },
16
17    "incremental": {
18      "steps": [
19        {
20          "step": 1,
21          "num_examples": 2,
22          "accumulated_dict": "...",
23          "accumulated_rules": "...",
24          "metrics": {
25            "corpus_chrfpp": 6.96
26          }
27        }
28      ]
29    }
30  }
31 }
```

Además del log detallado, el sistema genera un resumen compacto con las métricas principales de cada estrategia. Esta separación permite realizar comparaciones automáticas sin perder la información necesaria para auditar manualmente las respuestas.

5.2. Diseño de los prompts

5.2.1. Instrucción metalingüística base

Las estrategias M1 a M5 comienzan con una instrucción común en inglés que establece el marco de razonamiento:

You are an expert linguistic analyst performing inductive reasoning on an unknown language. You have never seen this language before. You must reason exclusively

from the provided examples using your meta-linguistic knowledge of how human languages work in general (word order typology, morphological patterns, agreement systems, affixation, etc.).

Core principles: Do NOT use any prior knowledge of this specific language or its family. Treat every hypothesis as provisional and subject to revision. Always show your reasoning explicitly before reaching any conclusion. A wrong rule confidently stated is worse than uncertainty acknowledged.

Varias decisiones de diseño en esta instrucción merecen justificación. Los prompts se escriben en inglés porque es la lengua en que los modelos tienen mejor rendimiento en tareas de razonamiento, y porque las traducciones del puzzle son siempre al inglés o desde el inglés. La instrucción de no usar conocimiento previo es necesaria porque los modelos más grandes pueden reconocer lenguas minoritarias que se suponen desconocidas, como se verificó con Lakota y Yoruba durante la selección de puzzles (véase la Sección ??). La instrucción de preferir la incertidumbre explícita a la confianza incorrecta reduce el número de casos en que el modelo genera traducciones inventadas con apariencia de corrección.

5.2.2. Formato de salida exigido

Todas las estrategias de traducción solicitan la respuesta en el mismo formato estructurado:

```

DICTIONARY:
[entradas morfema = significado, una por línea]

GRAMMAR RULES:
[reglas numeradas]

TRANSLATIONS:
1. [traducción]
2. [traducción]
```

Este formato tiene varias ventajas prácticas. Primero, la separación explícita entre diccionario, reglas y traducciones facilita la extracción automática de cada componente. Segundo, obligar al modelo a escribir el diccionario y las reglas antes de las traducciones le fuerza a hacer explícito su razonamiento, lo que funciona como un mecanismo de *chain-of-thought* implícito (Wei y cols., 2022). Tercero, la sección **TRANSLATIONS:** proporciona un marcador claro al extractor para localizar las hipótesis.

No obstante, los modelos no siempre respetan el formato exacto. El GPT-OSS-120B, en particular, tiende a intercalar análisis morfema a morfema dentro de la sección **TRANSLATIONS**, usando comillas tipográficas (") y flechas unicode (→) en lugar de líneas numeradas limpias. Este comportamiento motivó el diseño multicapa del extractor descrito en la Sección ??.

5.2.3. Prompt de M1 (baseline)

M1 proporciona todos los pares de entrenamiento en un único prompt. Su estructura es la más simple:

```

[instrucción metalingüística base]

---- TRAINING EXAMPLES ----
```

```

frase_1 →traducción_1
...
frase_n →traducción_n

---- YOUR TASK ----
Based on ALL examples above, write your inferred DICTIONARY
and GRAMMAR RULES, then [translate into English / translate
into LANG]:
1. frase_test_1
...

Format your response EXACTLY as:
DICTIONARY: ...
GRAMMAR RULES: ...
TRANSLATIONS: ...

```

5.2.4. Prompt de M2 (incremental)

M2 es la estrategia técnicamente más compleja porque el prompt cambia en cada paso e incluye la hipótesis acumulada del paso anterior. El prompt de cada paso k tiene la siguiente estructura:

```

[instrucción metalingüística base]

STEP k/total —New subset of examples:

---- YOUR CURRENT HYPOTHESIS (provisional —review critically) ----
DICTIONARY:
[diccionario acumulado del paso k-1]

GRAMMAR RULES:
[reglas acumuladas del paso k-1]

IMPORTANT: New examples may CONFIRM, REFINE or CONTRADICT
entries above. Delete proven—wrong entries; correct imprecise
ones.

---- NEW EXAMPLES ----
frase_i → traducción_i
...

---- YOUR TASK (in this order) ----

STEP A · ANALYSIS
[...]
Do NOT translate the test sentences yet.

STEP B · REVISED DICTIONARY
[...]
Mark deleted entries: [DELETED: entry —reason]

STEP C · REVISED GRAMMAR RULES
[...]

STEP D · TRANSLATIONS
[translate / traduce] —write ONLY the translation on each
line, nothing else:
1. frase_test_1
...

Format your response EXACTLY as:

```

```
ANALYSIS: ...
DICTIONARY: ...
GRAMMAR RULES: ...
TRANSLATIONS: ...
```

Tres elementos de diseño de este prompt merecen atención especial. La instrucción **Do NOT translate the test sentences yet** en el paso de análisis era necesaria porque en las primeras versiones el modelo anticipaba las traducciones en el paso de razonamiento, antes de haber actualizado el diccionario. La cláusula **[DELETED: entry - reason]** obliga al modelo a ser explícito sobre qué hipótesis descarta y por qué, en lugar de simplemente ignorar entradas anteriores, lo que produce un historial de razonamiento auditable. La instrucción **write ONLY the translation on each line, nothing else** fue necesaria para reducir la tendencia de los modelos a añadir notas entre paréntesis o justificaciones morfológicas junto a cada traducción.

5.2.5. Prompt de M3 (step-by-step)

M3 divide los ejemplos en cuatro etapas lingüísticas. El prompt de cada etapa incluye su foco específico:

```
STEP k/4 —Stage: LEXICAL / PHONOLOGY /
MORPHOSYNTAX / SYNTAX
[foco de la etapa]

---- ACCUMULATED KNOWLEDGE ----
DICTIONARY: [acumulado]
GRAMMAR RULES: [acumuladas]

---- EXAMPLES FOR THIS STAGE ----
frase_i →traducción_i
...

---- YOUR TASK ----
1. Refine DICTIONARY with new vocabulary
2. Add/update GRAMMAR RULES for this stage
3. [translate] using ALL accumulated knowledge
```

Las instrucciones de foco por etapa son: para léxico, *Focus on vocabulary and basic word order*; para fonología, *Focus on phonological patterns: vowel changes, consonant alternations, affixes, allomorphs*; para morfosintaxis, *Focus on agreement, tense, number, person, gender markings*; y para sintaxis, *Combine all rules: negation, questions, complex clauses*.

5.2.6. Prompt de M4 (inspiración humana)

M4 preponde el razonamiento humano del puzzle de Hakhun antes del puzzle nuevo, con instrucciones explícitas para no transferir léxico ni reglas:

```
[instrucción metalingüística base]

[razonamiento humano del puzzle de Hakhun]

Now apply the same style of reasoning to a NEW and COMPLETELY
DIFFERENT linguistic puzzle.
The language below is unrelated to the example above —do not
transfer any vocabulary or rules.
```

--- TRAINING EXAMPLES (new language) ---
[...]

El razonamiento humano incluye el análisis frase a frase del puzzle de Hakhun: qué elementos invariantes se identifican primero (**ne** como marcador de pregunta), cómo se aíslan los pronombres, cómo se deduce el sistema aspectual comparando terminaciones, y por qué **kəmə** se trata como marcador de transitividad sin traducción directa. Esto proporciona al modelo no solo el resultado final sino el proceso de razonamiento inductivo que llevó a él.

5.2.7. Prompt de M5 (autocorrección)

M5 opera en dos turnos. El primer turno usa el mismo prompt que M1. El segundo turno devuelve al modelo su propio output para que lo critique:

[instrucción metalingüística base]

You have already analyzed the following language and produced a first draft. Now act as a careful reviewer of your own work.

--- TRAINING EXAMPLES (same as before) ---
[...]

--- YOUR FIRST-DRAFT ANALYSIS ---
 DICTIONARY: [inferido en turno 1]
 GRAMMAR RULES: [inferidas en turno 1]
 FIRST-DRAFT TRANSLATIONS:
 1. [traducción del turno 1]
 ...

--- YOUR TASK ---
 1. CRITIQUE: For each translation above, check whether it is consistent with your dictionary and grammar rules. List specific contradictions or uncertainties. Be precise: quote the rule or entry that is violated or missing.
 2. REVISED DICTIONARY: Update entries if needed.
 3. REVISED GRAMMAR RULES: Update rules if needed.
 4. FINAL TRANSLATIONS: Produce corrected translations based on your critique. If a translation was already correct, keep it unchanged.

IMPORTANT: The TRANSLATIONS section must contain ONLY the final translated sentences, one per line, with no explanations, notes, or comments.

La instrucción de la sección **CRITIQUE** es deliberadamente precisa: el modelo debe citar la regla o entrada concreta que cada traducción viola. Sin esta precisión, los modelos tienden a producir críticas genéricas del tipo *the translation may not capture all morphological nuances*, que no conducen a ninguna corrección real. La instrucción **IMPORTANT** al final fue añadida tras observar que el GPT-OSS-120B incluía justificaciones morfológicas junto a cada traducción en el turno 2, lo que confundía al extractor.

5.3. Extracción automática de traducciones

La respuesta generada por un modelo no puede utilizarse directamente para calcular las métricas. Aunque los prompts solicitan un formato de salida específico, los modelos no siempre

lo respetan de forma estricta. En algunos casos añaden explicaciones, comentarios morfológicos o símbolos Markdown; en otros, mezclan las traducciones con entradas del diccionario o con instrucciones de revisión.

Por este motivo, se implementó un extractor específico cuya función es identificar únicamente las traducciones finales y descartar el resto del contenido. Este componente fue refinado progresivamente a medida que se detectaron nuevos formatos de respuesta durante los experimentos.

5.3.1. Patrones de salida observados

Durante el desarrollo se observaron varios patrones frecuentes. La Tabla 5.1 resume los principales casos y el tratamiento aplicado.

Tabla 5.1: Principales patrones detectados en las respuestas de los modelos.

Patrón	Ejemplo simplificado	Tratamiento
Línea numerada limpia	1. Do you sleep?	Extracción directa mediante un patrón de línea numerada.
Traducción precedida por flecha	n... -> ``Do you sleep?''	Extracción del contenido situado a la derecha de la flecha.
Nota final entre paréntesis	1. Do you sleep? (present tense)	Eliminación de la nota antes de calcular las métricas.
Entrada de diccionario	1. ne = question marker	Descarte mediante un filtro específico.
Meta-análisis lingüístico	1. The word order is SOV...	Descarte por contener terminología metalingüística.
Instrucción de revisión	1. Add a new dictionary entry...	Descarte mediante palabras clave asociadas a correcciones.
Salida incompleta	El modelo genera menos traducciones de las esperadas.	Registro explícito del fallo y penalización de las respuestas ausentes.

La variabilidad no depende únicamente del modelo utilizado. Una misma arquitectura puede cambiar su formato de salida entre estrategias o incluso entre pasos consecutivos de una estrategia incremental. Este comportamiento obliga a separar el procesamiento de las traducciones del razonamiento textual generado por el modelo.

5.3.2. Arquitectura multicapa del extractor

El extractor sigue un proceso secuencial formado por cuatro etapas:

1. Localizar la sección TRANSLATIONS de la respuesta.

2. Buscar traducciones situadas a la derecha de una flecha.
3. Buscar líneas numeradas cuando el patrón anterior no produce suficientes resultados.
4. Limpiar y filtrar los candidatos antes de devolver las traducciones.

El Código 5.4 muestra una versión simplificada de la implementación.

Código 5.4: Lógica principal del extractor de traducciones

```
1 def extract_translations(response, num_expected):
2     section = locate_translations_section(response)
3
4     arrow_hits = [
5         clean(match)
6         for match in find_arrow_translations(section)
7     ]
8
9     arrow_hits = [
10         text for text in arrow_hits
11         if is_valid_translation(text)
12     ]
13
14     if len(arrow_hits) >= num_expected:
15         return arrow_hits[:num_expected]
16
17     numbered_hits = [
18         clean(match)
19         for match in find_numbered_lines(section)
20     ]
21
22     numbered_hits = [
23         text for text in numbered_hits
24         if is_valid_translation(text)
25     ]
26
27     combined = arrow_hits + [
28         text for text in numbered_hits
29         if text not in arrow_hits
30     ]
31
32     return combined[:num_expected]
```

El primer patrón tiene prioridad porque algunos modelos incluyen una explicación morfema a morfema y terminan cada análisis mediante una flecha seguida de la traducción final. Si no se encuentran suficientes resultados, el sistema busca líneas numeradas, que constituyen el formato solicitado originalmente en los prompts.

Finalmente, ambos conjuntos se combinan eliminando duplicados y respetando el número de frases esperadas.

5.3.3. Filtros de descarte

No toda línea numerada representa una traducción. Especialmente en las estrategias M2 y M5, los modelos pueden numerar también reglas gramaticales, entradas de diccionario o instrucciones de autocorrección.

Para reducir estos falsos positivos se aplican varios filtros. El Código 5.5 resume su lógica.

Código 5.5: Filtros utilizados para descartar falsos positivos

```
1 def is_valid_translation(text):
2     if not 4 < len(text) < 120:
3         return False
4
5     if looks_like_dictionary_entry(text):
6         return False
7
8     if contains_revision_instruction(text):
9         return False
10
11    if contains_metalinguistic_analysis(text):
12        return False
13
14    return True
```

El filtro de entradas de diccionario detecta estructuras del tipo `palabra = significado` o `palabra: significado`. El filtro de instrucciones descarta líneas que contienen verbos como `add`, `delete`, `update` o `revise` junto con términos como `rule`, `dictionary` o `morpheme`. Por último, el filtro metalingüístico elimina explicaciones que incluyen expresiones como `word order`, `suffix`, `subject marker` o `allomorph`.

Estos filtros no sustituyen a una validación manual, pero reducen considerablemente el número de falsos positivos observados durante la ejecución automática.

5.3.4. Normalización de las traducciones

Antes de calcular las métricas, cada candidato pasa por una función de limpieza. Su objetivo es eliminar elementos de formato que no forman parte de la traducción: espacios innecesarios, comillas, marcas Markdown y notas explicativas situadas al final de la línea.

Una versión simplificada de esta función es la siguiente:

Código 5.6: Normalización básica de una traducción extraída

```
1 def clean(text):
2     text = text.strip()
3     text = text.strip("*")
4     text = text.strip("'")
5     text = remove_final_parenthetical_note(text)
6
7     return text.strip()
```

La limpieza debe aplicarse con cautela. Una eliminación excesivamente agresiva puede alterar una traducción válida. Por ejemplo, los marcadores `(sg)` y `(pl)` forman parte de algunas

referencias en inglés y no deben descartarse automáticamente. Por este motivo, únicamente se eliminan notas explicativas finales claramente separadas del contenido principal.

5.3.5. Gestión de respuestas incompletas

Durante los experimentos se detectaron casos en los que el extractor recuperaba menos traducciones que frases de test. Esta situación puede deberse a dos causas: el modelo no ha generado todas las respuestas solicitadas o el formato utilizado no ha sido reconocido correctamente.

Las traducciones ausentes no deben ignorarse durante el cálculo de métricas, ya que hacerlo favorecería artificialmente a las respuestas incompletas. Por este motivo, el sistema completa la lista con cadenas vacías hasta alcanzar el número de referencias esperadas.

El Código 5.7 muestra esta normalización.

Código 5.7: Penalización de traducciones ausentes

```
1 def complete_missing_translations(hypotheses, references):
2     hypotheses = list(hypotheses[:len(references)])
3
4     missing = len(references) - len(hypotheses)
5
6     if missing > 0:
7         hypotheses.extend([""] * missing)
8
9     return hypotheses
```

Además, el log conserva información explícita sobre el proceso de extracción:

Código 5.8: Metadatos asociados a la extracción

```
1 extraction_info = {
2     "num_expected": len(references),
3     "num_extracted": len(extracted_translations),
4     "extraction_complete":
5     len(extracted_translations) == len(references)
6 }
```

Este registro permite distinguir entre un error lingüístico del modelo y un problema de formato o extracción.

5.3.6. Validación del extractor

El extractor se validó mediante la revisión manual de respuestas generadas por los tres modelos evaluados y por las distintas estrategias. Para cada patrón detectado se comprobó que las traducciones recuperadas coincidieran con el contenido final producido por el modelo y que las líneas descartadas correspondieran realmente a explicaciones, instrucciones o entradas de diccionario.

La validación incluyó especialmente las respuestas de las estrategias iterativas, ya que generan un volumen mayor de texto y presentan más variabilidad de formato. También se revisaron los casos incompletos para determinar si el problema procedía de la generación del modelo o de una limitación del extractor.

Esta comprobación manual resulta necesaria porque las métricas automáticas solo son fiables cuando las hipótesis procesadas representan correctamente las traducciones generadas.

5.4. Construcción del conjunto de datos

El benchmark Linguini se distribuye en un único archivo JSONL que contiene problemas de distintos tipos. Para facilitar la ejecución automática de los experimentos, se implementó un proceso de conversión que transforma cada problema seleccionado en un archivo JSON independiente con una estructura homogénea.

Esta conversión cumple dos objetivos. En primer lugar, separa claramente los pares utilizados como contexto de las frases reservadas para evaluación. En segundo lugar, permite ejecutar nuevos experimentos sin depender del formato interno del benchmark original.

5.4.1. Extracción desde el JSONL de Linguini

El archivo original de Linguini contiene 160 problemas. Cada registro incluye, entre otros, los campos `context`, `query`, `answer`, `task_lang`, `task_type` y `eval_type`.

El conversor procesa cada registro mediante los siguientes pasos:

1. **Filtrado por tipo de tarea.** Se conservan únicamente los problemas de traducción, identificados mediante el campo `task_type = translation`. Tras este filtrado permanecen 114 de los 160 problemas originales.
2. **Extracción de los pares de entrenamiento.** El campo `context` contiene ejemplos en texto libre. Cada línea se procesa para obtener un par formado por una expresión de origen y su traducción.
3. **Extracción de las frases de test.** El campo `query` contiene las expresiones que el modelo debe traducir. Estas frases se separan de las instrucciones generales del enunciado.
4. **Normalización de las respuestas.** El campo `answer` puede contener una única traducción válida o varias alternativas. Cuando existen múltiples referencias, el sistema conserva la primera como referencia principal para el cálculo automático de métricas.
5. **Inferencia de la dirección de traducción.** Se distingue entre problemas de traducción desde la lengua desconocida al inglés (`lang2en`) y problemas en la dirección inversa (`en2lang`).
6. **Generación del archivo JSON.** Para cada problema válido se crea un archivo independiente con los campos necesarios para la ejecución del sistema experimental.

El Código 5.9 muestra una versión reducida de la estructura generada.

Código 5.9: Estructura simplificada de un puzzle convertido

```
1{
2  "id": "linguini_012018020100",
3  "language": "Hakhun",
```

```
4 "language_code": "nst-hkn_Latn",
5 "language_family": "Sino-Tibetan",
6 "task_type": "translation",
7 "translation_direction": "lang2en",
8
9 "train_pairs": [
10 {
11 "source": "ŋ'a ka k ne",
12 "target": "Do I go?"
13 }
14 ],
15
16 "test_pairs": [
17 {
18 "source": "n ip ku ne",
19 "target": "Do you(sg) sleep?"
20 }
21 ]
22 }
```

Los pares incluidos en `train_pairs` se incorporan al prompt como contexto. Por el contrario, las traducciones almacenadas en `test_pairs` se utilizan únicamente después de recibir la respuesta del modelo y nunca se incluyen durante la inferencia.

5.4.2. Criterios de selección

No todos los problemas de traducción resultan adecuados para el diseño experimental planteado. Tras la conversión inicial se aplican varios criterios de selección:

Suficiencia de ejemplos. El puzzle debe contener al menos ocho pares de entrenamiento y tres frases de test. Esta condición garantiza que la estrategia incremental pueda generar varios pasos y que las métricas no dependan de una única respuesta.

Diversidad lingüística. Se priorizan lenguas pertenecientes a familias distintas con el objetivo de ampliar la variedad tipológica del conjunto evaluado.

Dirección de traducción. Se incluyen problemas en ambas direcciones: desde la lengua desconocida al inglés y desde el inglés a la lengua desconocida. Esta distinción permite estudiar por separado la comprensión y la generación de morfología no familiar.

Ausencia de referencias vacías. Se descartan los problemas que contienen respuestas ausentes o listas vacías en el conjunto de test.

Conversión válida. Se excluyen los registros en los que una instrucción del enunciado original ha sido interpretada erróneamente como una frase que debe traducirse.

Control exploratorio de conocimiento previo. La estrategia M0 se utiliza para detectar posibles indicios de conocimiento previo de la lengua. Este control no permite demostrar que el modelo desconozca por completo el idioma, pero ayuda a identificar casos que requieren una revisión manual adicional.

La selección final combina criterios automáticos con una comprobación manual de los casos dudosos. Esta revisión es necesaria porque algunos errores no pueden detectarse de forma fiable mediante una única regla general.

5.4.3. Validación de los puzzles convertidos

Durante el desarrollo se observaron algunos errores de conversión provocados por el formato heterogéneo de los enunciados originales. El caso más frecuente consistía en interpretar una instrucción parcial como si fuera la primera frase de test. Por ejemplo, una indicación incluida en el enunciado podía acabar almacenada como una expresión que el modelo debía traducir.

Para reducir estos errores, cada puzzle convertido se valida mediante varias comprobaciones:

1. El número de frases de test debe coincidir con el número de referencias disponibles.
2. Ninguna referencia puede estar vacía.
3. Las frases de test no deben comenzar con expresiones típicas de una instrucción, como `translate`, `spoken on`, `given the following` o construcciones equivalentes.
4. Los registros dudosos deben revisarse manualmente antes de incluirse en la selección final.

El Código 5.10 resume estas comprobaciones.

Código 5.10: Validación básica de un puzzle convertido

```
1 def validate_puzzle(puzzle):
2     test_pairs = puzzle["test_pairs"]
3
4     if len(test_pairs) < 3:
5         return False
6
7     for pair in test_pairs:
8         source = pair["source"].strip()
9         target = pair["target"].strip()
10
11     if not source or not target:
12         return False
13
14     if looks_like_instruction(source):
15         return False
16
17     return True
```

Los puzzles descartados se registran junto con la causa del rechazo. Esta información facilita la trazabilidad del proceso de selección y permite revisar posteriormente los casos excluidos.

5.5. Problemas encontrados y soluciones

Durante el desarrollo del sistema se detectaron distintas incidencias relacionadas con las APIs externas, el procesamiento de las respuestas y el cálculo de métricas. Algunas se re-

solvieron mediante modificaciones en el código, mientras que otras constituyen limitaciones inherentes al entorno experimental.

En esta sección se resumen los problemas más relevantes y las decisiones adoptadas para reducir su impacto.

5.5.1. Limitaciones de las APIs y los proveedores

Los experimentos se ejecutan mediante servicios externos gratuitos. Esta decisión permite evaluar modelos de distintos tamaños sin disponer de infraestructura propia, pero introduce restricciones que deben tenerse en cuenta al interpretar los resultados.

La Tabla 5.2 resume las principales incidencias observadas.

Tabla 5.2: Principales incidencias relacionadas con las APIs utilizadas.

Incidencia	Consecuencia	Solución adoptada
Límites de tasa de las APIs	Algunas peticiones eran rechazadas temporalmente mediante respuestas HTTP 429.	Se añadieron reintentos automáticos con tiempos de espera crecientes y pausas entre llamadas consecutivas.
Latencia elevada en modelos grandes	Algunas peticiones superaban el tiempo de espera establecido, especialmente en estrategias iterativas.	Se amplió el tiempo máximo de espera y se registraron los errores sin detener la ejecución de las estrategias restantes.
Límite diario de solicitudes	Algunos proveedores restringen el número máximo de peticiones gratuitas que pueden realizarse en un día.	Los experimentos se distribuyeron en varias sesiones y se almacenaron resultados parciales para evitar repetir ejecuciones ya completadas.
Respuestas vacías o incompletas	En algunas llamadas el proveedor devolvía una respuesta sin contenido utilizable o con menos traducciones de las solicitadas.	Se añadieron comprobaciones explícitas y se penalizaron las respuestas ausentes durante el cálculo de métricas.
Dependencia de servicios externos	La disponibilidad de un modelo o su comportamiento puede cambiar con el tiempo.	Se registraron el proveedor, el identificador del modelo, la fecha de ejecución y los parámetros utilizados en cada experimento.

La función responsable de las llamadas a las APIs incorpora un mecanismo de reintento cuando se alcanza temporalmente el límite de uso. El Código 5.11 muestra una versión simplificada.

Código 5.11: Reintentos automáticos tras alcanzar el límite de tasa

```

1 for attempt in range(3):
2     response = requests.post(
3         provider_url,
```

```
4 headers=headers,  
5 json=payload,  
6 timeout=timeout  
7 )  
8  
9 if response.status_code == 429:  
10 wait = 30 * (attempt + 1)  
11 time.sleep(wait)  
12 continue  
13  
14 response.raise_for_status()  
15 return parse_response(response)  
16  
17 raise RuntimeError("Persistent rate limit after 3 attempts")
```

Este mecanismo no elimina completamente los fallos de red o de disponibilidad, pero evita que una limitación puntual invalide automáticamente toda la ejecución.

5.5.2. Problemas de implementación y extracción

La extracción automática de traducciones fue la parte que requirió más ajustes durante el desarrollo. Los modelos no siempre respetaban el formato solicitado y podían mezclar respuestas, reglas gramaticales, entradas de diccionario e instrucciones de revisión.

La Tabla 5.3 recoge los problemas más relevantes.

Estos problemas muestran que el cálculo automático de métricas no puede separarse del análisis del formato de salida. Una traducción ausente o una línea extraída de forma incorrecta alteraría directamente el resultado cuantitativo.

Por este motivo, además de los filtros automáticos, se realizó una revisión manual de una muestra de respuestas de los distintos modelos y estrategias.

5.5.3. Problemas relacionados con las métricas

La elección de métricas también requirió ajustes durante el desarrollo. Los puzzles evaluados contienen frases cortas y, en algunos casos, diferencias morfológicas pequeñas. Estas características hacen que algunas métricas habituales de traducción automática resulten poco informativas cuando se interpretan de forma aislada.

La Tabla 5.4 resume las principales decisiones.

El uso conjunto de varias métricas permite distinguir distintos tipos de error. Exact match identifica las traducciones completamente correctas. chrF++ refleja mejor las coincidencias parciales a nivel de caracteres y palabras. TER mide el número de ediciones necesarias para transformar la hipótesis en la referencia. BLEU se conserva principalmente para facilitar la comparación con trabajos anteriores.

5.5.4. Limitaciones que permanecen abiertas

Algunas limitaciones no pueden eliminarse completamente mediante cambios en el código.

En primer lugar, el sistema depende de proveedores externos. Incluso con temperatura cero, una misma petición puede no producir exactamente la misma respuesta en momentos

Tabla 5.3: Principales problemas detectados durante la extracción de respuestas.

Problema	Efecto observado	Solución adoptada
Extracción del lado incorrecto de una flecha	En algunos formatos se recuperaba la expresión original en lugar de la traducción.	Se ajustó el patrón para conservar únicamente el contenido situado a la derecha del símbolo.
Confusión entre traducciones y entradas de diccionario	Líneas como <code>ne = question marker</code> podían interpretarse como respuestas finales.	Se añadió un filtro específico para detectar asignaciones mediante signos <code>=</code> o <code>:</code> .
Inclusión de notas morfológicas	Algunas líneas contenían explicaciones sobre sufijos, morfemas o estructura sintáctica.	Se incorporó un filtro basado en términos metalingüísticos frecuentes.
Inclusión de instrucciones de revisión	En M5 podían recuperarse frases como <code>Update this rule</code> o <code>Delete this entry</code> .	Se añadieron patrones que detectan verbos de corrección junto con términos como <code>rule</code> , <code>dictionary</code> o <code>morpheme</code> .
Caracteres fonéticos no reconocidos	Algunos patrones iniciales no detectaban correctamente caracteres IPA.	Se amplió el rango Unicode admitido por las expresiones regulares.
Eliminación excesiva de paréntesis	La función de limpieza podía modificar traducciones válidas con marcas como <code>(sg)</code> o <code>(pl)</code> .	Se restringió la eliminación a notas explicativas finales claramente separadas de la traducción.
Respuestas incompletas	En algunos pasos se extraían menos traducciones de las esperadas.	Se añadieron metadatos de extracción y las respuestas ausentes se completaron con cadenas vacías para penalizarlas correctamente.

distintos si el proveedor modifica internamente el modelo o la infraestructura de inferencia.

En segundo lugar, el control M0 solo permite detectar indicios de conocimiento previo. Una traducción incorrecta sin contexto no demuestra que el modelo desconozca completamente la lengua, del mismo modo que una coincidencia parcial no demuestra necesariamente que la conozca.

En tercer lugar, las métricas automáticas no sustituyen al análisis manual. Dos traducciones formalmente distintas pueden ser lingüísticamente equivalentes, y una respuesta con chrF++ elevado puede contener un error gramatical relevante.

Por estos motivos, los resultados cuantitativos se complementan con un análisis cualitativo de casos representativos.

Tabla 5.4: Problemas detectados durante el cálculo e interpretación de métricas.

Problema	Consecuencia	Decisión adoptada
Valores BLEU muy bajos	BLEU asigna puntuaciones reducidas cuando las frases son cortas o difieren en una única palabra.	Se mantiene por compatibilidad con la literatura, pero no se utiliza como métrica principal.
Confusión entre chrF y chrF++	La configuración inicial podía calcular chrF estándar sin incorporar n-gramas de palabras.	Se fijó explícitamente el parámetro <code>word_order=2</code> para obtener chrF++.
Penalización de variantes parcialmente correctas	Una respuesta puede ser semánticamente cercana a la referencia y fallar únicamente en número gramatical, tiempo verbal o persona.	Se combinan chrF++, BLEU, TER y exact match para obtener una visión más completa.
Evaluación parcial de respuestas incompletas	Si solo se evaluaban las traducciones extraídas, una salida incompleta podía obtener una puntuación artificialmente alta.	Se completan las traducciones ausentes con cadenas vacías antes de calcular las métricas.
Selección optimista del mejor paso	Elegir el mejor paso de una estrategia incremental mediante las respuestas de test introduce un sesgo favorable.	El último paso se utiliza como resultado principal. El mejor paso se conserva únicamente como información diagnóstica sobre la evolución del modelo.

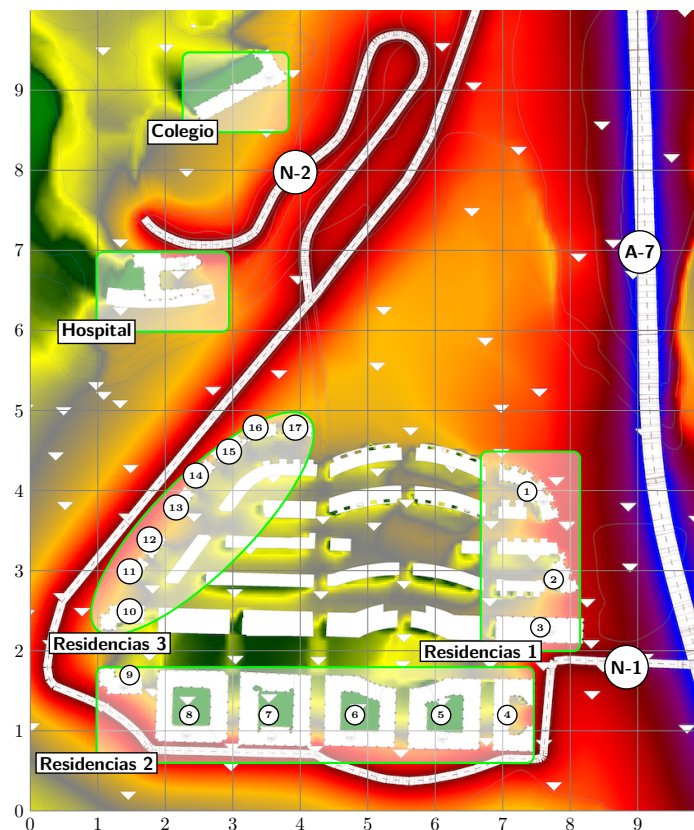
6. Resultados (Con ejemplos de gráficos)

6.1. Diagramas

Gracias al paquete *Tikz* se pueden incluir multitud de medios gráficos, diagramas, capas sobre imágenes, etc. Existen múltiples formas de realizarlo, para ello es recomendable consultar la guía de iniciación disponible aquí: <http://cremeronline.com/LaTeX/minimaltikz.pdf> y también el manual completo disponible aquí: <http://osl.ugr.es/CTAN/graphics/pgf/base/doc/pgfmanual.pdf>.

A continuación se muestran algunos ejemplos. Revisa el archivo .tex para ver cómo se utilizan.

Imagen a la que se le ha añadido cuadros y texto desde latex:



En muchas ocasiones es necesario realizar un diagrama de bloques, más abajo se muestra

un ejemplo de ello. En la red hay multitud de ejemplos que pueden ser fácilmente modificables para un fin concreto, como por ejemplo en esta web: <http://www.texample.net/tikz/examples/tag/block-diagrams/>.

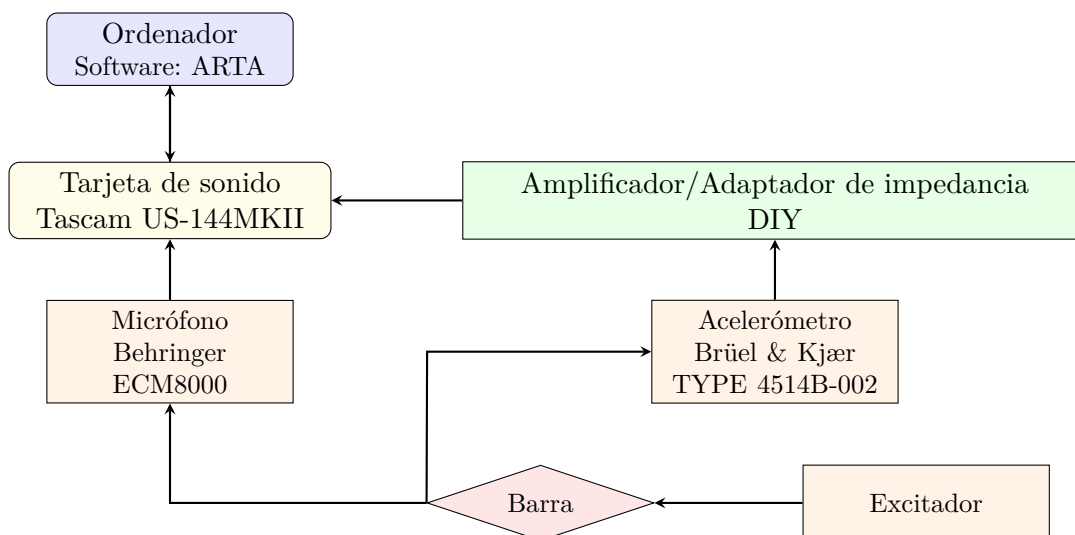


Figura 6.1: Diagrama realizado en latex con Tikz.

6.2. Gráficas

Existen múltiples formas de generar gráficas para latex. Hay disponibles herramientas como GeoGebra que dispone de la utilidad para exportar los gráficos en formato Tkiz. También funciones para Matlab que genera las gráficas que muestra habitualmente pero en código para Tkiz.

6.2.1. Línea

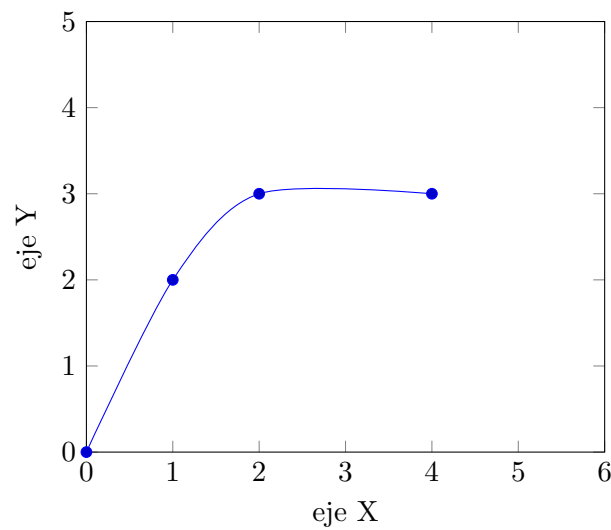
La forma más simple, aunque no sencilla cuando abarca muchos datos es la siguiente:

```

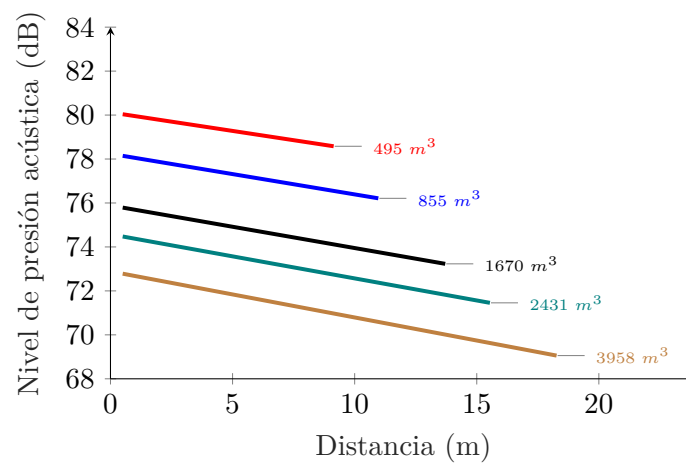
\begin{figure}[ht]
\centering
\begin{tikzpicture}
\begin{axis}
[ymin=0,ymax=5, % Límites del eje y
xmin=0,xmax=6, % Límites del eje x
ylabel= eje Y, % Nombre del eje y
xlabel= eje X] % Nombre del eje x
\addplot+[smooth] coordinates % Une los puntos curva suavizada
{(0,0) (1,2) (2,3 (4,3))}; % Puntos de la gráfica
\end{axis}
\end{tikzpicture}
\caption{Gráfica sencilla.}
\end{figure}

```

El resultado es el siguiente:

**Figura 6.2:** Gráfica sencilla.

Otro ejemplo, en este caso las líneas están calculadas directamente en LaTeX y después cada una tiene una anotación (el código se encuentra en el archivo `archivos/ejemplos/perjudicialesoptica-centro.tex`).

**Figura 6.3:** OP/S003

6.2.2. Barras

Otro ejemplo es la gráfica de barras:

```
\begin{figure}[ht]
\centering
\begin{tikzpicture}
\begin{axis}[
ybar=12pt,
ymin=0,ymax=150,
xtick=data,
```

```

enlarge x limits={abs=2cm},
symbolic x coords={rubio, moreno},
bar width = 20pt,
ylabel= número,
xlabel= color de pelo,
ytick align=outside,
ytick pos=left,
major x tick style = transparent,
legend style={at={(0.04,0.96)},anchor=north west, font=\footnotesize, legend cell align=left,},
]
\addplot[ybar,fill=blue, area legend] coordinates {
(rubio,20)
(moreno,100)};
\addplot[ybar,fill=purple, area legend] coordinates {
(rubio,110)
(moreno,105)};
\legend{Chicos, Chicas}
\end{axis}
\end{tikzpicture}
\caption{Gráfica barras.}
\end{figure}

```

El resultado es el siguiente:

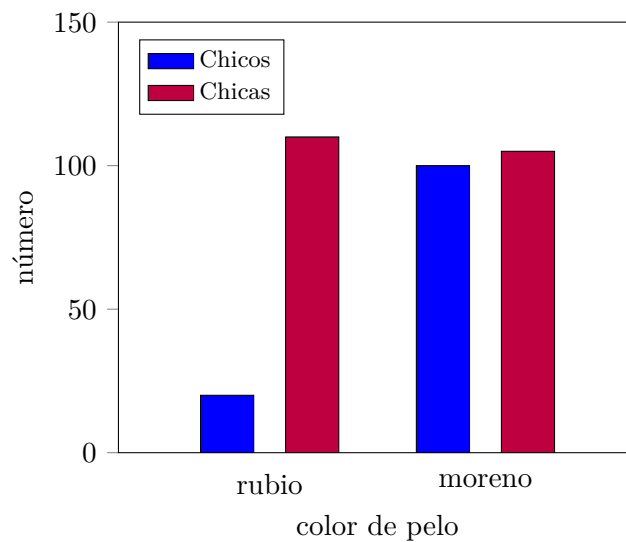


Figura 6.4: Gráfica barras.

6.2.3. Polar

Un ejemplo de gráfica polar semicircular (ver archivo `archivos/ejemplos/polarnorm.tex`):

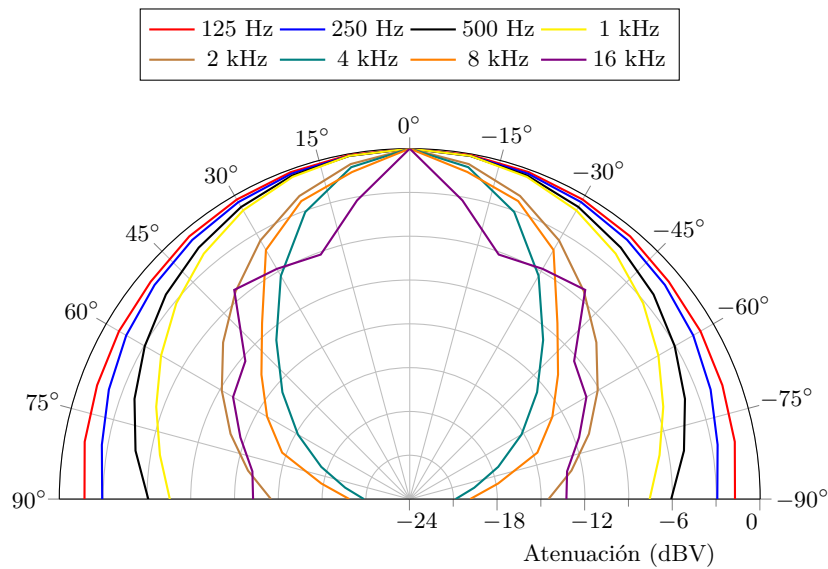


Figura 6.5: Directividad normalizada del altavoz (0 dBV en el eje).

6.3. Importados de MATLAB

Gracias a la herramienta *matlab2tikz* (<https://es.mathworks.com/matlabcentral/fileexchange/22022-matlab2tikz-matlab2tikz>) se pueden exportar las gráficas de cualquier tipo de Matlab a latex. Después de incluir los archivos de *matlab2tikz* se debe escribir una llamada después de crear la figura tal que:

Código 6.1: Ejemplo de llamada a *matlab2tikz*

```
1 fig = plot(x,y);
2 matlab2tikz('figurehandle',fig,'NombreArchivo.tex','height','5cm','width','13.5cm','strict',true,'↔
  ↳ showHiddenStrings',true,'showInfo',false)
```

Y para utilizar el archivo generado por la función en este documento:

```
\begin{figure}[ht]
\centering
{\scalefont{0.8}\input{archivos/ejemplos/ParedFina} }
\caption{Ejemplo de gráfica obtenida con matlab2tikz.}
\end{figure}
```

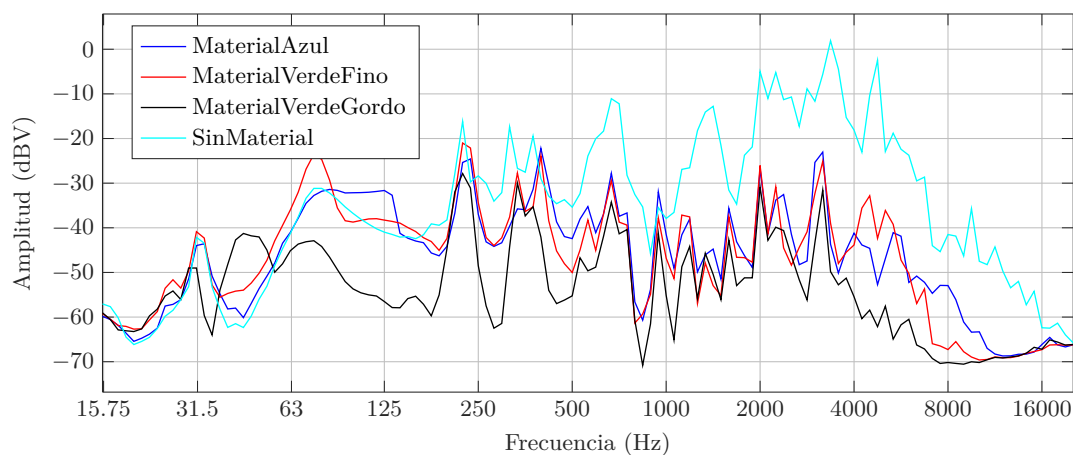


Figura 6.6: Ejemplo de gráfica obtenida con *matlab2tikz*.

Ejemplo de una gráfica 3D generada en Matlab y exportada por *matlab2tikz*:

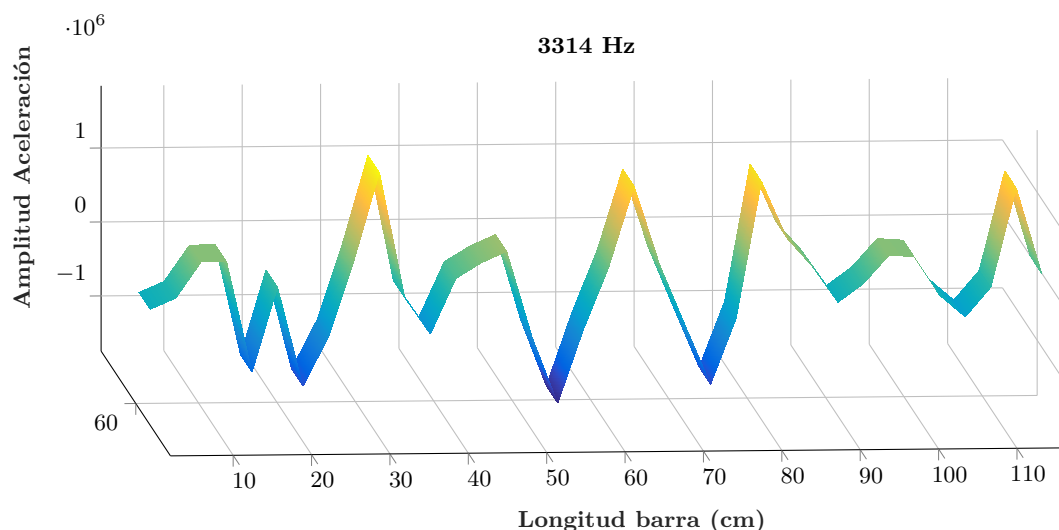


Figura 6.7: Amplitud de la aceleración en el modo número 8.

6.4. Ejemplo avanzado

El potencial del paquete *Tikz* es muy alto, se pueden realizar muchísimas cosas. En la red se facilitan muchos ejemplos para poder ver el funcionamiento y aprender. Existen hilos donde la gente publica sus mejores diseños de *Tikz* como en <https://tex.stackexchange.com/questions/158668/nice-scientific-pictures-show-off> o páginas donde facilitan muchas plantillas como <http://www.texample.net/tikz/examples/all/>.

Un ejemplo de lo que se puede llegar a conseguir es el siguiente:

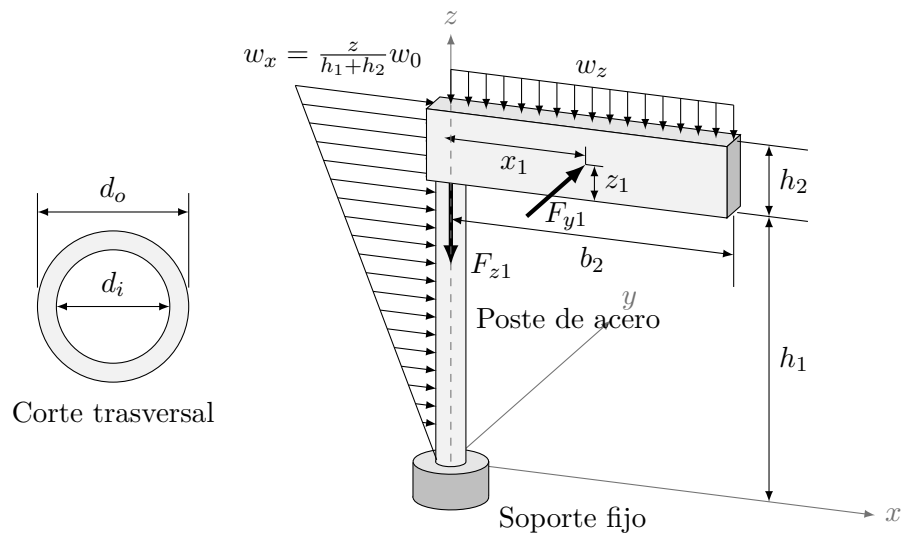


Figura 6.8: Señal realizada con Tikz, sin imágenes.

7. Conclusiones (Con ejemplos de matemáticas)

7.1. Matemáticas

En $\text{\LaTeX}$ se pueden mostrar ecuaciones de varias formas, cada una de ellas para un fin concreto.

Antes de ver algunas de estas formas hay que conocer cómo se escriben fórmulas matemáticas en $\text{\LaTeX}$. Una fuente de información completa es la siguiente: <https://en.wikibooks.org/wiki/LaTeX/Mathematics>. También existen herramientas online que permiten realizar ecuaciones mediante interfaz gráfica como <http://www.hostmath.com/>, <https://www.mathcha.io/editor> o <https://www.latex4technics.com/>

Para mostrar una ecuación numerada se debe utilizar:

```
\begin{equation}
\nabla\times{\mathbf H}=\left[\frac{1}{r}\frac{\partial}{\partial r}(rH_\theta)-\frac{1}{r}\frac{\partial H_r}{\partial\theta}\right]\hat{\mathbf z}
\label{ecuacion}
\end{equation}
```

$$\nabla \times \mathbf{H} = \left[\frac{1}{r} \frac{\partial}{\partial r} (r H_\theta) - \frac{1}{r} \frac{\partial H_r}{\partial \theta} \right] \hat{\mathbf{z}} \quad (7.1)$$

Si es necesario agrupar varias ecuaciones en un mismo índice se puede escribir del siguiente modo:

```
\begin{subequations}
\begin{eqnarray}
{\mathbf E}&=&E_z(r,\theta)\hat{\mathbf z}\label{ecu1} \\
{\mathbf H}&=&H_r(r,\theta)\hat{\mathbf r}+H_\theta(r,\theta)\hat{\bm \theta}\label{ecu2}
\end{eqnarray}
\end{subequations}
% Se incluye '&' entre la igualdad para centrar las ecuaciones desde el '='.
```

$$\mathbf{E} = E_z(r, \theta) \hat{\mathbf{z}} \quad (7.2a)$$

$$\mathbf{H} = H_r(r, \theta) \hat{\mathbf{r}} + H_\theta(r, \theta) \hat{\boldsymbol{\theta}} \quad (7.2b)$$

Otras dos formas que son las habituales en muchos lugares para incluir ecuaciones son:

Ejemplo de fórmula en línea con el texto `\int_{a}^{b} f(x)dx = F(b) - F(a)`, esta ecuación quedará dentro $\leftrightarrow$ del texto.

Esta otra, al utilizar dos `'$'`, se generará en una línea nueva `\int_{a}^{b} f(x)dx = F(b) - F(a)`

Ejemplo de fórmula en línea con el texto $\int_a^b f(x)dx = F(b) - F(a)$, esta ecuación quedará dentro del texto.

Esta otra, al utilizar dos `'$'`, se generará en una línea nueva

$$\int_a^b f(x)dx = F(b) - F(a)$$

También se puede añadir información adicional a una ecuación con la función *condiciones* creada para esta plantilla:

```
\begin{equation}
\underset{z=z_0}{\mathrm{Res}}\{f(z)\}=\frac{1}{(m-1)!}\lim_{z\rightarrow z_0}\left[\frac{d^{m-1}}{dz^{m-1}}\left[(z-z_0)^m f(z)\right]\right]
\end{equation}

\begin{condiciones}[donde:]
% Excepto 'Descripción y valor' el resto no es necesario el símbolo $para texto matemático.
% Item & Relación & Descripción o valor
m & \rightarrow & Es la multiplicidad del polo $z_0$ \\
z_0 & \rightarrow & Es la parte que se iguala a 0 con el polo. \\
f(z) & \rightarrow & Es la función contenida en la integral.
\end{condiciones}
```

$$\mathrm{Res}(f(z)) = \frac{1}{(m-1)!} \lim_{z \rightarrow z_0} \left[\frac{d^{m-1}}{dz^{m-1}} [(z - z_0)^m f(z)] \right] \quad (7.3)$$

donde: $m \rightarrow$ Es la multiplicidad del polo z_0

$z_0 \rightarrow$ Es la parte que se iguala a 0 con el polo.

$f(z) \rightarrow$ Es la función contenida en la integral.

Si lo que deseas es una ecuación alineada a la izquierda o derecha puedes hacerlo con lo siguiente (el `'&'` simple es utilizado para alinear las ecuaciones desde ese punto, los iguales):

```
% Alineado a la izquierda al incluir al final el doble '&&'
\begin{flalign}
y_{h_1} &= & \begin{bmatrix} 6\cos(\sqrt{6}x) \\ -\sqrt{6}\sin(\sqrt{6}x) \end{bmatrix} e^x & \& \& \\
y_{h_2} &= & \begin{bmatrix} 6\sin(\sqrt{6}x) \\ \sqrt{6}\cos(\sqrt{6}x) \end{bmatrix} e^x & \& \& \\
\end{flalign}

% Alineado a la derecha al incluir al inicio el doble '&&'
\begin{flalign}
&\& \& y_{h_1} = & \begin{bmatrix} 6\cos(\sqrt{6}x) \\ -\sqrt{6}\sin(\sqrt{6}x) \end{bmatrix} e^x \\
&\& y_{h_2} = & \begin{bmatrix} 6\sin(\sqrt{6}x) \\ \sqrt{6}\cos(\sqrt{6}x) \end{bmatrix} e^x \\
\end{flalign}
```

$$y_{h_1} = \begin{bmatrix} 6 \cos(\sqrt{6}x) \\ -\sqrt{6} \sin(\sqrt{6}x) \end{bmatrix} e^x \quad (7.4)$$

$$y_{h_2} = \begin{bmatrix} 6 \sin(\sqrt{6}x) \\ \sqrt{6} \cos(\sqrt{6}x) \end{bmatrix} e^x \quad (7.5)$$

$$y_{h_1} = \begin{bmatrix} 6 \cos(\sqrt{6}x) \\ -\sqrt{6} \sin(\sqrt{6}x) \end{bmatrix} e^x \quad (7.6)$$

$$y_{h_2} = \begin{bmatrix} 6 \sin(\sqrt{6}x) \\ \sqrt{6} \cos(\sqrt{6}x) \end{bmatrix} e^x \quad (7.7)$$

Tanto con la función utilizada en (7.1,7.3), como en (7.2a,7.2b) y en las anteriores, si se les incluye un '*' después de 'equation', 'subequation' o 'flalign', se elimina la numeración de las ecuaciones pero manteniendo el resto de características.

Bibliografía

- AI@Meta. (2024). *Llama 3 Model Card*. <https://github.com/meta-llama/llama3>.
- Bean, A. M., Hellsten, S., Mayne, H., Magomere, J., Chi, E. A., Chi, R., ... Kirk, H. R. (2024). LINGOLY: A Benchmark of Olympiad-Level Linguistic Reasoning Puzzles in Low-Resource and Extinct Languages. En *Advances in neural information processing systems (neurips 2024), track on datasets and benchmarks*. Descargado de <https://arxiv.org/abs/2406.06196>
- Bozhanov, B., y Derzhanski, I. (2013). Rosetta Stone Linguistic Problems. En *Proceedings of the fourth workshop on teaching nlp and cl* (pp. 1–8). Sofia, Bulgaria: Association for Computational Linguistics.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... others (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. Descargado de <https://arxiv.org/abs/2005.14165>
- Chi, N., Malchev, T., Kong, R., Chi, R., Huang, L., Chi, E., ... Radev, D. (2024). Modeling: A Novel Dataset for Testing Linguistic Reasoning in Language Models. En *Proceedings of the 6th workshop on research in computational linguistic typology and multilingual nlp* (pp. 113–119). St. Julian's, Malta: Association for Computational Linguistics.
- Choudhary, M., Srivatsa, K. A., Aeron, G., Bhattacharya, A. R., Dinh, D. K. D., Hanif, I. A., ... Choudhary, M. (2025). unweiLing: What Makes Linguistics Olympiad Puzzles Tricky for LLMs? En *Proceedings of the conference on language modeling (colm 2025)*. Descargado de <https://arxiv.org/abs/2508.11260> (Analiza 629 problemas en 41 lenguas de baja densidad. Concluye que la complejidad morfológica es el factor más determinante del fracaso de los LLMs)
- Derzhanski, I., y Payne, T. E. (2010). The Linguistics Olympiads: Academic Competitions in Linguistics for Secondary School Students. En *Linguistics at school: Language awareness in primary and secondary education* (pp. 213–226). Cambridge University Press.
- Eberhard, D. M., Simons, G. F., y Fennig, C. D. (2020). *Ethnologue: Languages of the World*. Dallas, Texas: SIL International. Descargado de <https://www.ethnologue.com> (23rd edition)
- Joshi, P., Santy, S., Budhiraja, A., Bali, K., y Choudhury, M. (2020). The State and Fate of Linguistic Diversity and Inclusion in the NLP World. En *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 6282–6293). doi: 10.18653/v1/2020.acl-main.560

- Majmudar, N., y Filatova, E. (2025). Can LLMs Solve and Generate Linguistic Olympiad Puzzles? *arXiv preprint arXiv:2509.21820*. Descargado de <https://arxiv.org/abs/2509.21820> (Trabajo más reciente del campo. Demuestra que o1 supera a humanos en la mayoría de tipos de puzzle excepto en sistemas de escritura y lenguas poco estudiadas)
- Papineni, K., Roukos, S., Ward, T., y Zhu, W.-J. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. En *Proceedings of the 40th annual meeting of the association for computational linguistics* (pp. 311–318). doi: 10.3115/1073083.1073135
- Popović, M. (2015). chrF: Character n-gram F-score for Automatic MT Evaluation. En *Proceedings of the tenth workshop on statistical machine translation* (pp. 392–395). Lisbon, Portugal: Association for Computational Linguistics. doi: 10.18653/v1/W15-3049
- Popović, M. (2017). chrF++: Words Helping Character n-grams. En *Proceedings of the second conference on machine translation* (pp. 612–618). Copenhagen, Denmark: Association for Computational Linguistics. doi: 10.18653/v1/W17-4726
- Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. En *Proceedings of the third conference on machine translation: Research papers* (pp. 186–191). Association for Computational Linguistics. doi: 10.18653/v1/W18-6319
- Ramji, R., y Ramji, K. (2024). Inductive Linguistic Reasoning with Large Language Models. *arXiv preprint arXiv:2412.17819*. Descargado de <https://arxiv.org/abs/2412.17819> (Propone prompting analógico: el modelo genera ejemplos auxiliares de lenguas relacionadas para guiar el razonamiento inductivo)
- Sánchez, E., Alastruey, B., Ropers, C., Stenetorp, P., Artetxe, M., y Costa-jussà, M. R. (2024). Linguini: A Benchmark for Language-Agnostic Linguistic Reasoning. En *arxiv preprint arxiv:2409.12126*. Descargado de <https://arxiv.org/abs/2409.12126> (Meta AI / University College London / Universidad del País Vasco)
- Tanzer, G., Suzgun, M., Visser, E., Jurafsky, D., y Melas-Kyriazi, L. (2024). A Benchmark for Learning to Translate a New Language from One Grammar Book. En *Proceedings of the 12th international conference on learning representations (iclr 2024)*. Descargado de <https://arxiv.org/abs/2309.16575>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., ... Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35. Descargado de <https://arxiv.org/abs/2201.11903>
- Zhu, H., Liang, Y., Xu, W., y Xu, H. (2025). Evaluating Large Language Models for In-Context Learning of Linguistic Patterns in Unseen Low Resource Languages. En *Proceedings of the first workshop on language models for low-resource languages (lo-reslm) at acl 2025*. Association for Computational Linguistics. Descargado de <https://aclanthology.org/2025.loreslm-1.31>
- Şahin, G. G., Kementchedjhieva, Y., Rust, P., y Gurevych, I. (2020). PuzzLing Machines: A Challenge on Learning From Small Data. En *Proceedings of the 58th annual meeting*
-

of the association for computational linguistics (pp. 1241–1254). Online: Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.115

A. Anexo I

Aquí vendría el anexo I

B. Páginas horizontales

Aquí se muestra cómo incluir páginas en horizontal.
Esta página está en vertical

Esta página está en horizontal

Esta página también está en horizontal

Esta página está de nuevo en vertical

C. Importar PDF

A continuación se muestra una página importada de un PDF externo. Observar los comentarios en el código de este anexo para más información. También puedes leer el manual con todas las opciones en <http://osl.ugr.es/CTAN/macros/latex/contrib/pdfpages/pdfpages.pdf>.

Alicante

15 DE MARZO DE 2007

Expediente número

Referencia del peticionario

AYUNTAMIENTO DE ALICANTE

Departamento de Medio Ambiente

C/San Nicolás, nº 2, 4º

03001 ALICANTE

Contacto: Juan Luís Beresaluze

DOCUMENTO DE SÍNTESIS

***ELABORACIÓN DEL MAPA ACÚSTICO MUNICIPAL DE LA CIUDAD DE
ALICANTE***

Fecha de realización del estudio: MAYO 2005 – MARZO 2007